Nafs-Aql: A Verifier-Gated Dual-Process Architecture on Small Qwen Models for CPU-Only Deployment

Dirac R&D Team

***Abstract*—We present Nafs-Aql, a dual-process language-model system that runs on four CPU cores with no GPU. A 1.7B Qwen3 model answers at once; a 4B reasoning model is consulted only when a gate, derived from expected loss, judges that the error it is expected to remove is worth the waiting time. Where an answer can be checked (mathematics, code, structured data, Lean proofs) an exact verifier decides, and every reply carries an assurance level. We give a formal model whose results fix the design: the optimal gate is a threshold rule on a calibrated error estimate; the precision of a verified answer is bounded by the verifier's false-accept rate, not by model strength; unverified self-refinement converges to a fixed point that can be worse than the first draft, while a verifier-gated loop has risk linear in the false-accept rate; and plurality voting cannot repair confidently wrong answers. The system is a single Rust gateway with its own inference engine, a verifier-scored agent harness for code, a knowledge base that reads PDF pages by their layout and searches an in-process lexical and vector index with a graph built without a model call, a rule-based check of each answer against its sources, and two memories that need no model, one of verified answers and one of earlier conversations. On a shared Intel i3-8100, with the verifier visible the gate matched reasoning-only accuracy at 0.36 of its cost; a hybrid search over 847 passages takes 0.15 s, of which the search itself is 0.2 ms; 18 of 20 held-out questions were answered from passages against 0 to 2 without; a follow-up question finds its passage first ten times as often when it is read with the turn before it; and reading PDF pages by their layout, with vectors for each passage's sentence windows, puts the answering passage first for 0.875 of quoted questions against 0.768 and 0.386 of two-passage questions against 0.284. We report with equal weight what did not pay, including speculative decoding, 1-bit weights, concurrent hedging and a difficulty signal read from the question alone.**

***Index Terms*—dual-process reasoning, test-time compute, routing, verification, small language models, CPU inference, retrieval-augmented generation, agent harness, Rust.**

I. Introduction

Reasoning-tuned language models answer hard questions by spending more computation at inference time: they sample long chains of thought, draw several candidates, and verify or vote among them [1]–[4]. The gain is real but expensive, and it is wasted on the many requests that a direct answer would have handled [5], [6]. This is the engineering form of an old observation about cognition: there is a fast, automatic mode and a slow, effortful one, and competence lies in knowing when to switch [7]–[9].

Manuscript of November 2026. Contact: Dirac R&D Team. All measurements were taken on one four-core machine shared with other workloads (Section X); sample sizes are small and are stated with each result.

A. Problem

Small models are fast and wrong too often. Large models are careful and need a GPU or a remote service. Neither says which of its answers to trust. We ask what can be built under a hard constraint: pretrained, openly licensed Qwen models of at most 4B parameters, a machine with four CPU cores and no accelerator, and nothing leaving that machine. Under this constraint a reasoning model decodes 5 to 8 tokens per second, so a deliberate answer takes minutes; the system must spend that time only where it buys correctness, and must be able to say when it did not get a checked answer.

B. Inspiration

The name joins two terms from the classical philosophy of mind of the Islamic intellectual tradition: *nafs*, the impulsive self that acts first, and *'aql*, the faculty of reason that restrains and corrects it. We use the pair only as a naming analogy for a fast process and a deliberate one, in the same spirit as the System 1 and System 2 of dual-process psychology. Nothing in the design depends on the analogy, and every component has a plain engineering name.

C. Contributions

1) A formal model of a verifier-gated dual-process system with seventeen stated results, each with explicit assumptions and the design decision it fixes (Section III).
2) An architecture in which acceptance needs evidence: an expected-loss gate with certified per-bin risk, exact verifiers whose verdict is final, bounded loops, and an assurance level on every reply (Sections IV and V).
3) A verifier-scored agent harness for small models: a best-first candidate tree, an append-only transcript that keeps the engine's prompt cache valid, a per-step decoding grammar that leaves the reasoning span free, and a cascade that tries the quick model first (Section VI).
4) A CPU inference layer, including an engine that serves several users in one pass over the weights with output identical to a solo run, and a measured account of which forms of parallelism pay on four cores (Section VII).
5) A knowledge base that reads PDF pages by their layout (columns, headings, tables, running headers) with no model, searches documents by words and meaning from an in-process index, scores each passage by its best sentence window as well as its whole text, builds a graph of a document set without a model call, answers with numbered

sources and says which sentences of an answer its sources hold; and two memories that need no model, one of verified answers and one of earlier conversations (Sections VIII and IX).

6) Measurements of all of the above on one shared machine, with interleaved comparisons and the negative results reported beside the positive ones (Section X), and a comparison with related open systems (Section XI).

D. What this paper does not claim

The quality results are indicative: comparisons use tens to a few hundred items, far below what a release decision needs, and every time was measured on a machine shared with other workloads. No public retrieval benchmark was run, so the knowledge results are measured changes against our own earlier versions, not against published systems. Section XII lists what remains open.

II. Related Work

A. Dual-process theory and metareasoning

Dual-process accounts distinguish a fast, automatic, associative mode from a slow, effortful, rule-based one [7], [9], [10]; Evans and Stanovich treat the question of *when* the slow mode intervenes as central [8]. The expected-value-of-control theory models that allocation as a cost–benefit decision [11], resource-rational analysis makes the same move for cognition in general [12], and the metareasoning literature asks how an agent should decide which computations to run [13]. Decision neuroscience adds sequential forms: evidence accumulated to a bound [14], optimal as Wald's sequential probability ratio test [15], [16], with a bound that collapses under a cost of time [17], [18]. Theorem 2 is the one-step instance of these ideas, and Section V-D uses the sequential ones.

B. Test-time compute and self-correction

Chain-of-thought prompting elicits multi-step reasoning [1]; sampling several chains and voting improves accuracy [2]. Coverage grows roughly log-linearly in the number of samples, but without an automatic verifier the gain from voting plateaus [19]; trained verifiers recover much of it [20], [21]. Allocating compute per prompt by difficulty is several times as efficient as uniform best-of-N [3]. Reinforcement learning induces long reasoning traces [4], [22], [23], budget forcing closes them [24], and Qwen3 ships both modes in one model with a thinking budget [25]. Self-critique helps on some tasks [26], [27] but without external feedback does not improve reasoning and can degrade it [28], [29]; Theorem 4 gives the simplest model in which this happens.

C. Deciding when to think, routing and cascades

A growing literature trains or prompts models to choose between a direct answer and a reasoning trace per query [6], [30]–[34], or stops a trace early on a confidence signal [35]–[37]. Routing between a cheap and an expensive model on predicted difficulty [38], [39] and cascading on the weak model's consistency [40], [41] reduce cost at matched quality. The classical foundations are Chow's reject option [42], selective classification with risk guarantees [43] and learning to defer [44], [45]; calibration of the routing signal is the crux [46]–[48]. vLLM Semantic Router [49] is a production decision layer in front of many models; Section XI compares it with our gate.

D. Agent harnesses

SWE-agent showed that an edit-time check raises the resolve rate of a coding agent [50]; Agentless that selection by tests beats majority voting [51]; observation masking that old tool output can be dropped at no loss [52]. Concise execution feedback is what lets models repair code [53], and best-first search over verified candidates is competitive with heavier tree search when the verifier is exact [54]. We draw mechanisms from Open Math Harness [55], the core of OpenAI's Codex CLI [56] and DeepSeek Harness [57]. Small models are worse at diff formats than at whole-file rewrites [58], [59], and constraining the format of the reasoning itself can cost accuracy [60].

E. CPU inference

Speculative decoding drafts with a cheap model and verifies in parallel [61], [62]; Medusa and EAGLE replace the draft model with extra heads [63], [64]; model-free variants draft from earlier text [65]. Batching across requests and prefix caching are the basis of modern serving [66]–[68], and hedged requests trade compute for tail latency [69]. Low-bit formats such as LittleBit [70] and table-lookup kernels [71] change the model rather than the engine. Section X-H reports which of these pay on four cores.

F. Retrieval, graphs and memory

Hybrid retrieval fuses lexical and dense rankings, commonly by reciprocal rank [72], and indexing a passage with its context improves both [73]. Layout analysis decides what a passage is in the first place: LiteParse [74] reads PDF pages by recursive XY-cut over positioned glyphs, with headings, tables and running headers found by geometry and no model, and semtools [75] searches with static embeddings of single lines and returns a window around each hit. Graph-based retrieval builds an entity graph over passages; OntoSmith [76] governs such a graph with a pinned ontology. Memorilla [77] compresses a document collection into a few vectors that a frozen decoder reads as input tokens. Prompt injection through retrieved text is a first-class threat [78], [79].

III. Mathematical Formulation

The objects of the model are policies, verifiers, costs and losses. Each result ends with its *engineering consequence*: the parameter or the algorithm of the system that it fixes. Results are labelled by the strength of their assumptions: **[exact]** holds under the stated model without further hypotheses; **[strong]** needs an assumption that must be checked empirically.

A. Objects

Definition 1 (Task, answer, correctness)**.** A *task* is $x \in \mathcal{X}$ (a query together with its conversational context). An *answer* is $y \in \mathcal{Y}$. Correctness is an unobserved indicator $\kappa(x, y) \in \{0, 1\}$. A task is *verifiable* if a deterministic procedure can evaluate κ (unit tests, numeric or symbolic equality, schema validity); otherwise it is *non-verifiable*.

Definition 2 (Fast and slow paths)**.** The *fast path* (System 1) is a stochastic policy $\pi_1(y \mid x)$ with expected cost $c_1(x)$ and correctness probability $p_1(x) = \mathbb{P}_{y\sim\pi_1}[\kappa(x, y) = 1]$. The *slow path* (System 2, deliberation) is a policy $\pi_2(y \mid x, b)$ parametrised by a *thinking budget* $b \in \mathbb{N}$ (maximum reasoning tokens), with cost $c_2(x, b)$ and correctness $p_2(x, b)$. Costs are measured in a common unit (seconds of CPU time or tokens weighted by a tariff).

Definition 3 (Verifier)**.** A *verifier* is a map $V : \mathcal{X}\times\mathcal{Y} \to \{0, 1\}$ with true-accept rate $r = \mathbb{P}[V = 1 \mid \kappa = 1]$ and false-accept rate $\varphi = \mathbb{P}[V = 1 \mid \kappa = 0]$. A verifier is *exact* if $\varphi = 0$ and $r = 1$ on its domain. Soft verifiers (rubric judges, consensus) have $\varphi > 0$ and must publish measured (r, φ).

Definition 4 (Gate and trace state)**.** After the fast path has run, the *controller* observes a feature vector $s = \sigma(x, \text{trace}_1) \in \mathcal{S}$ (token log-probabilities, agreement among cheap samples, verifier outcome, domain class) and chooses, with probability $\gamma(s) \in [0, 1]$, to *escalate* to the slow path. The lifecycle of one request is a finite automaton over the *trace states* $\mathcal{Z} = \{\mathsf{A}, \mathsf{L}, \mathsf{M}, \mathsf{T}, \mathsf{X}\}$ of Fig. 2: A (an unverified draft), L (the draft has been flagged by the gate or a verifier), M (a deliberate retry conditioned on evidence), T (an accepted answer) and X (abstention with an explicit uncertainty notice). The automaton is an ordinary bounded retry loop.

Definition 5 (Losses and objective)**.** A wrong answer costs $L > 0$, an abstention costs $L_a \in (0, L)$, and compute is priced by $\lambda > 0$ per cost unit. For a controller $\theta = (\gamma, b, N, R)$ (gate, thinking budget, samples per round, retry rounds) the design objective is

$$\mathcal{J}(\theta) = \mathbb{E}_x\big[L\,\mathbb{P}[\text{wrong} \mid x, \theta] + L_a\,\mathbb{P}[\text{abstain} \mid x, \theta] + \lambda\,\mathbb{E}[\text{cost} \mid x, \theta]\big]. \tag{1}$$

Lemma 1 (Termination)**.** [exact] *With at most N slow-path samples per round and at most R retry rounds, every request makes at most $1 + N(R+1)$ model calls and at most $1 + N(R+1)$ verifier calls before reaching* T *or* X*.*

Proof. Each transition of the automaton either consumes one of the $N(R+1)$ slow samples or terminates; the fast path is called once. □ □

Consequence. Every request has a hard upper bound on latency; the gateway enforces N and R as configuration limits rather than letting the model decide when to stop.

B. Precision of a verified answer

Theorem 1 (Verified-return precision)**.** [exact] *Let a candidate be correct with prior probability p and let it be accepted by a verifier with rates (r, φ). Then*

$$\rho(p) \;=\; \mathbb{P}[\textit{wrong} \mid \textit{accepted}] = \frac{\varphi(1-p)}{\varphi(1-p) + r\,p}, \tag{2}$$

and $\rho(p) \le \varepsilon$ if and only if $\varphi \le \dfrac{\varepsilon\, r\, p}{(1-\varepsilon)(1-p)}$. If k verifiers are applied and their acceptances are conditionally independent given κ, the same holds with $\varphi \leftarrow \prod_i \varphi_i$ and $r \leftarrow \prod_i r_i$.

Proof. Bayes' rule with $\mathbb{P}[\text{accepted}] = \varphi(1-p) + rp$. Solving $\rho \le \varepsilon$ for φ gives the bound. Conditional independence factorises the joint acceptance probabilities. □ □

Consequence. The precision of the system on verifiable tasks is bounded by the verifier's false-accept rate, not by the strength of either model. Every verifier publishes (r, φ) measured by mutation testing on a held-out set, and the per-verifier acceptance threshold is 1.0 for exact verifiers. For hard tasks ($p = 0.1$) a 1% precision target requires $\varphi \le 1.1 \times 10^{-3}$: only exact verifiers qualify, which is why consensus (Theorem 6) is a flag and not a verifier.

C. The gate is a threshold rule

Let $e_1(s) = \mathbb{P}[\text{fast answer wrong} \mid s]$, $e_2(s) = \mathbb{P}[\text{final answer wrong after escalation} \mid s]$ and $\Delta c(s) = \mathbb{E}[\text{extra cost of escalation} \mid s]$.

Theorem 2 (Optimal gate)**.** [exact]

(i) The gate minimising (1) *escalates if and only if*

$$L\,\big(e_1(s) - e_2(s)\big) \;>\; \lambda\,\Delta c(s), \tag{3}$$

with ties broken arbitrarily.

(ii) Under a budget constraint $\mathbb{E}[\gamma(s)\Delta c(s)] \le B$ instead of the price λ, the optimal gate escalates if and only if $\big(e_1(s) - e_2(s)\big)/\Delta c(s) > \lambda^\star$, where $\lambda^\star$ is the smallest multiplier that meets the budget; randomisation is needed only on the level set $\{s : (e_1 - e_2)/\Delta c = \lambda^\star\}$.

(iii) If $e_1 - e_2$ is non-decreasing in a scalar score s and Δc is constant, the rule is a single threshold $s > \tau$.

Proof. (i) Conditional on s, keeping costs $Le_1(s)$ and escalating costs $\lambda\Delta c(s) + Le_2(s)$; the objective is an expectation over s of a pointwise choice, so pointwise minimisation is optimal. (ii) The Lagrangian $\mathbb{E}[\gamma(s)(Le_2 + \lambda\Delta c - Le_1)]$ is linear in γ; by the Neyman–Pearson lemma the optimal γ is an indicator of a likelihood-ratio set with randomisation on the boundary. (iii) Immediate from (i). □ □

Consequence. The controller is implemented as a calibrated estimator $\hat{e}_1(s)$ (logistic or isotonic regression on the fast-path features), a per-domain estimate $\hat{e}_2$, and the comparison (3). The only tunable is the cost ratio $\lambda\Delta c/L$ (or equivalently the budget B), which the operator sets; the threshold is *derived*, not hand-tuned. The rule coincides with Chow's optimal reject rule [42] and with the expected-value-of-control account of cognitive control [11], with "reject" read as "deliberate".

Proposition 1 (Price of miscalibration)**.** [exact] *If the plug-in gate uses estimates with $\sup_s |\hat{e}_1(s) - e_1(s)| \le \epsilon_1$ and*

$\sup_s |\hat{e}_2(s) - e_2(s)| \le \epsilon_2$, *its objective exceeds that of the optimal gate by at most* $L(\epsilon_1 + \epsilon_2)$.

Proof. At any s the two gates disagree only if $|L(e_1 - e_2) - \lambda\Delta c| \le L(\epsilon_1+\epsilon_2)$, and the loss from a wrong decision equals that absolute difference. Taking expectations over s gives the bound. □ □

Consequence. The calibration target of the gate is set from the regret the operator tolerates: a maximum calibration error of 0.05 on $\hat{e}_1$ bounds the excess loss by $0.05L$ per request.

Proposition 2 (Cost–quality frontier)**.** [exact] *The set of achievable pairs (expected cost, expected error) over all gates γ is convex, and its lower-left boundary is traced by the family of threshold gates of Theorem 2 as λ varies.*

Proof. Both coordinates are linear in γ, so mixtures of gates achieve mixtures of pairs; the boundary of a convex set is supported by linear functionals, each of which is minimised by a threshold gate by Theorem 2. □ □

Consequence. The user-facing "speed–care" control in the chat UI is a single scalar λ; no other knob can reach a better point.

D. Deliberation: sampling, stopping and retries

Let a slow-path round draw one candidate and verify it; let $a = q\,r + (1-q)\varphi$ be the probability that a draw is accepted, where q is the probability that a draw is correct.

Theorem 3 (Coverage and cost of sequential verified sampling)**.** [exact] *With at most M draws and stopping at the first acceptance, (i) the probability that some candidate is accepted is $1-(1-a)^M$; (ii) the expected number of draws is $E(M) = \big(1-(1-a)^M\big)/a$; (iii) the conditional error of the accepted candidate is $\rho(q)$ of Theorem 1, independent of M; (iv) if a varies across tasks with mean $\bar{a}$, the coverage is at most $1-(1-\bar{a})^M$.*

Proof. (i)–(ii) are the truncated geometric distribution. (iii) Each accepted candidate, whichever draw produced it, is wrong with probability $\rho(q)$ by Bayes' rule. (iv) $u \mapsto (1-u)^M$ is convex, so Jensen's inequality gives $\mathbb{E}[(1-a)^M] \ge (1-\bar{a})^M$. □ □

Consequence. More samples buy *coverage*, never *precision*: the number of draws is chosen from a coverage target and the per-task heterogeneity of a must be measured, not averaged.

Proposition 3 (Optimal number of draws under uncertainty about a)**.** [strong: Beta prior, monotone stopping] *Suppose the acceptance probability of a task is unknown with prior $a \sim \mathrm{Beta}(\alpha,\beta)$, each draw costs c_p, and failing to produce an accepted candidate costs L_a. After m consecutive rejections the posterior mean is $a_m = \alpha/(\alpha+\beta+m)$. The policy that draws again if and only if $a_m L_a > c_p$ is optimal, so the optimal cap is*

$$M^\star = \left\lceil \frac{\alpha L_a}{c_p} - \alpha - \beta \right\rceil \tag{4}$$

(the number of $m \ge 0$ with $\alpha/(\alpha+\beta+m) > c_p/L_a$), when the right-hand side is positive, and $M^\star = 0$ otherwise.

Proof. A further draw costs c_p and, with posterior-predictive probability a_m, saves L_a; so the one-step look-ahead rule continues iff $a_m L_a > c_p$. Because every continuation happens only after a rejection and a_m is decreasing in m, once the rule says "stop" it says "stop" in all subsequent states: the problem is monotone, and in the monotone case the one-step look-ahead rule is optimal [80]. The draws made are exactly those indexed by the $m \ge 0$ with $m < \alpha L_a/c_p - \alpha - \beta$, whose count is the ceiling. □ □

Consequence. The retry limit R and the samples per round N (with $M = N(R+1)$) are derived per domain from a fit of (α,β) on baseline runs and the operator's cost ratio c_p/L_a. The Beta model is a modelling choice, to be checked against the empirical distribution of acceptance rates.

E. Self-monitoring: why the loop must be verifier-gated

Self-monitoring is a loop in which a draft is re-examined and amended. Let u be the probability that one unverified refinement turns a wrong answer into a right one and d the probability that it turns a right answer into a wrong one.

Theorem 4 (Unverified refinement converges to a fixed point)**.** [strong: homogeneous Markov refinement] *If q_t is the probability that the t-th draft is correct and $q_{t+1} = q_t(1-d)+(1-q_t)u$, then $q_t = q^\star + (q_0 - q^\star)(1-u-d)^t$ with $q^\star = u/(u+d)$. In particular q_t decreases monotonically in t whenever $q_0 > q^\star$.*

Proof. The recursion is affine with fixed point $q^\star$ and contraction factor $1-u-d \in (-1,1)$ for $u+d \in (0,2)$. □ □

Consequence. Iterating self-critique without external evidence can only help tasks whose first draft is worse than the fixed point; for a strong first draft it *harms*. This is consistent with the empirical finding that language models do not reliably self-correct without external feedback [28], [29]. Nafs-Aql therefore never accepts an unverified refinement: on non-verifiable tasks the loop is capped at $R = 1$.

Theorem 5 (Verifier-gated loop is stable)**.** [exact] *In a loop that accepts a candidate only when a verifier with rates (r,φ) passes it, and whose t-th candidate is correct with probability q_t, the probability of terminating with an accepted wrong answer is at most $\varphi\,\mathbb{E}[T]$, where T is the number of candidates examined, and the error conditional on acceptance lies in $[\min_t \rho(q_t), \max_t \rho(q_t)]$. Neither bound depends on the refinement dynamics (u,d).*

Proof. A wrong candidate at round t is accepted with probability φ; summing $\mathbb{P}[\text{reach } t]\,(1-q_t)\varphi \le \varphi\mathbb{P}[\text{reach } t]$ over t gives $\varphi\,\mathbb{E}[T]$. Conditional on acceptance at round t the error is $\rho(q_t)$ by Theorem 1; the overall conditional error is a mixture of these values. □ □

Consequence. With a verifier in the loop the risk is linear in φ and in the expected number of rounds, both of which the controller bounds (Lemma 1); the refinement prompt can be changed freely without re-deriving the safety case.

F. Consensus has a floor

Theorem 6 (Plurality voting converges to the modal answer)**.** [exact] *Let N answers be drawn i.i.d. from $\pi(\cdot \mid x)$ over a finite answer set with a unique mode $y^\circ(x)$. Then the plurality vote converges almost surely to $y^\circ(x)$ as $N \to \infty$, so the limiting accuracy of self-consistency is $\mathbb{P}_x[\,y^\circ(x)$ is correct$]$. For finite N, if the correct answer has mass q and the strongest wrong answer has mass $\bar{q} < q$, the probability that the wrong answer ties or beats the correct one is at most $\exp\big(-N(q-\bar{q})^2/2\big)$.*

Proof. The empirical frequencies converge almost surely to $\pi(\cdot \mid x)$ (strong law), so the arg-max converges to the unique mode. For the finite bound let $D_i = \mathbb{1}[y_i = y^\star] - \mathbb{1}[y_i = \bar{y}] \in [-1, 1]$ with mean $q - \bar{q}$; Hoeffding's inequality [81] gives $\mathbb{P}[\sum_i D_i \le 0] \le \exp(-2N(q-\bar{q})^2/4)$. □ □

Consequence. Agreement among samples cannot repair tasks on which the model is confidently wrong; the mass of such tasks is a floor that no N removes. Consensus is therefore recorded as an *assurance level* (`consensus`) distinct from `verified`, is never used to write training data without a human check, and the number of samples for a consensus flag is set from the measured margin $q - \bar{q}$.

G. Independence of the two paths

Proposition 4 (Anchoring and collision)**.** [exact] *If the slow path does not see the fast draft, the probability that both produce the same wrong answer is the collision probability $\sum_{y:\kappa(x,y)=0} \pi_1(y \mid x)\pi_2(y \mid x)$. Passing the draft to the slow path can only add anchoring on top of this value. Certifying an adoption rate below 5% at confidence 95% with zero observed adoptions needs at least $\lceil \ln 0.05 / \ln 0.95 \rceil = 59$ trials; with observed adoptions the Clopper–Pearson interval [82] applies.*

Consequence. The slow path is *draft-blind* by default; draft-conditioned deliberation is an ablation, and a coupling test measures the adoption rate. Because both paths may share weights (the single-model profile of Section IV-D), the collision term is not small a priori and is measured.

H. Honest self-knowledge under routing

Proposition 5 (Unbiased capability estimates need audits)**.** [exact] *Slow-path correctness is observed only on escalated tasks, so the naive estimate is $\mathbb{P}[$slow correct $\mid$ escalated$]$, a biased estimate of p_2. If a random fraction $\epsilon > 0$ of the kept tasks is also run through the slow path, then with escalation rate g, $\hat{p}_2 = g\,\hat{p}_{2|\mathrm{esc}} + (1-g)\,\hat{p}_{2|\mathrm{audit}}$ is unbiased, with standard error at most $\frac{1}{2}\sqrt{\big(g + (1-g)/\epsilon\big)/n}$ over n requests.*

Proof. Inverse-probability weighting with known strata weights g and $1-g$ and sampling probabilities 1 and ϵ; the variance of each stratum mean is at most $1/(4\,n_{\text{stratum}})$ with $n_{\text{esc}} = ng$ and $n_{\text{audit}} = n(1-g)\epsilon$. □ □

Consequence. The self-model (the audit ledger) is built only from traces plus a 5% audit stream; capabilities with no audited data are reported as *unmeasured*, never as zero. With $n = 10^4$, $g = 0.3$ and $\epsilon = 0.05$ the standard error is at most 0.019.

I. Estimation and gate sample sizes

Proposition 6 (Sample sizes)**.** [exact]

(i) (Threshold from a quantile.) If τ is set as the empirical $(1-g^\star)$-quantile of n i.i.d. scores, the realised escalation rate is within ε of $g^\star$ with probability at least $1-\delta$ whenever $n \ge \ln(2/\delta)/(2\varepsilon^2)$ [83].

(ii) (Non-inferiority.) To show that the cascade loses at most m points of accuracy against the slow-only baseline with power $1-\beta$ at level α, a paired design with discordance rate δ needs $n \ge (z_\alpha + z_\beta)^2\,\delta/m^2$ pairs.

(iii) (Binomial gates.) Certifying a rate below ε with zero failures at confidence $1-\delta$ needs $n \ge \ln\delta / \ln(1-\varepsilon)$ trials.

Consequence. A baseline that fixes the escalation rate to within two points needs at least $4\,600$ tasks ($\varepsilon = 0.02$, $\delta = 0.05$); a one-point non-inferiority claim needs about $6\,200$ pairs at $\delta = 0.1$ (or 700 for a three-point margin). With $n = 500$ the escalation rate is known only to ± 6 points. The measurements of Section X are below these sizes and are labelled indicative for that reason.

J. Distillation without self-deception

Proposition 7 (Label noise and verifier-inflated gains)**.** [exact]

(i) If a fraction η of training labels is wrong, then for every hypothesis h, $|\mathrm{err}_{noisy}(h) - \mathrm{err}_{clean}(h)| \le \eta$; with verifier-filtered traces $\eta = \rho$ of Theorem 1.

(ii) If accuracy is measured with a verifier of rates (r, φ), the measured rate is $\varphi + (r-\varphi)\,t$ where t is the true rate; a measured gain Δ corresponds to a true gain $\Delta/(r-\varphi)$, and training on verifier-accepted outputs can raise φ and inflate the measurement.

(iii) A paired promotion gate with discordance δ resolving a gain of ε needs $n \ge z^2\delta/\varepsilon^2$ pairs per domain.

Proof. (i) The two error events differ only on examples whose label is wrong. (ii) Expand $\mathbb{P}[V=1] = r\,t + \varphi(1-t)$. (iii) Normal approximation to the paired binomial. □ □

Consequence. Promotion of a distilled fast-path adapter requires a confidence-bound gain on a verifier that was never used for data collection, plus a per-domain non-regression bound and human sign-off.

K. What a CPU can serve

Proposition 8 (Decode and prefill ceilings)**.** [exact given the roofline model] *For a dense decoder whose active weights occupy W bytes and whose key–value cache at context length ℓ occupies $K(\ell)$ bytes, the steady-state decode rate on a machine with memory bandwidth β is at most $\beta/(W + K(\ell))$ tokens per second, and the prefill rate on a machine with peak throughput F FLOP/s and P active parameters is at most $F/(2P)$ tokens per second.*

Proof. Each decoded token must stream every active weight and the cache from memory at least once; each prefill token requires at least $2P$ floating-point operations. □ □

Consequence. On the reference machine (dual-channel DDR4-2400, $\beta \approx 38\,\text{GB/s}$; AVX2, four cores) a 4-billion-parameter model in 4-bit GGUF ($W \approx 2.5\,\text{GB}$) cannot exceed about 15 tokens/s, a 1.7B model about 35 tokens/s, and a 1000-token prefill of the 4B model takes seconds to tens of seconds. These ceilings fix the model sizes, the thinking-budget cap, the context cap and the prompt-cache requirement. Measured values on the reference machine (Section VII) are below the ceilings, as expected.

Proposition 9 (Thinking budget)**.** [strong: concave returns] *If $p_2(x,b)$ is concave and non-decreasing in b and the cost is linear, $c_2(x,b) = c_2^0 + \kappa_{tok}\, b$, the budget minimising $L(1 - p_2(x,b)) + \lambda c_2(x,b)$ is the unique $b^\star$ with $L\,\partial_b p_2(x,b^\star) = \lambda\kappa_{tok}$ (or the boundary if no interior solution exists).*

Consequence. The thinking budget is a per-domain configuration value estimated from the accuracy-versus-budget curve; when the budget is exhausted the controller forces the answer by closing the reasoning span [24] rather than returning nothing.

L. End-to-end identity

Proposition 10 (Cascade difference)**.** [exact] *Let ρ_1 be the error on tasks the gate keeps and $\bar{e}_2^{kept}$ the error the slow path alone would make on those tasks. If the slow path behaves identically on escalated tasks in both systems, $\mathbb{P}[\textit{cascade wrong}] - \mathbb{P}[\textit{slow-only wrong}] = (1-g)\big(\rho_1 - \bar{e}_2^{kept}\big)$.*

Consequence. Non-inferiority within m requires $\rho_1 \le \bar{e}_2^{\text{kept}} + m/(1-g)$; $\bar{e}_2^{\text{kept}}$ is observable only through the audit stream of Proposition 5, which is therefore part of the safety case.

M. Summary of engineering consequences

Table I lists each result with the parameter or algorithm it determines.

TABLE I
FORMAL RESULTS AND WHAT THEY DETERMINE

Result	Engineering consequence
Lemma 1	Hard caps N, R; latency bound
Thm. 1	Verifier (r,φ) publication; exact-verifier threshold 1.0
Thm. 2	Gate = calibrated $\hat{e}_1$, $\hat{e}_2$, cost ratio; no hand-tuned τ
Prop. 1	Calibration-error target of the gate
Prop. 2	Single "speed–care" scalar in the UI
Thm. 3	N from coverage target; measure heterogeneity of a
Prop. 3	R, N from Beta fit and c_p/L_a
Thm. 4	No unverified refinement; $R = 1$ on non-verifiable tasks
Thm. 5	Loop risk $\le \varphi\,\mathbb{E}[T]$; prompt changes need no new safety case
Thm. 6	Consensus is an assurance flag; N from margin
Prop. 4	Draft-blind slow path; coupling test
Prop. 5	5% audit stream; "unmeasured" label
Prop. 6	Baseline size $\ge 4\,600$; sample sizes of claims
Prop. 7	Promotion gate: held-out verifier, confidence bounds
Prop. 8	Model sizes, context cap, prompt cache, budget cap
Prop. 9	Per-domain thinking budget; budget forcing
Prop. 10	Audit stream is part of the safety case

IV. ARCHITECTURE

A. Design principles

Four principles follow from Section III. (1) *The gate is derived, not tuned*: the controller compares a calibrated error estimate with a cost ratio (Theorem 2). (2) *Acceptance needs evidence*: no answer is marked verified unless an exact verifier accepted it; consensus and judge scores are assurance levels, not proofs (Theorems 1 and 6). (3) *Every loop is bounded*: N, R and the thinking budget are configuration limits enforced by the gateway (Lemma 1). (4) *Self-knowledge is audited*: the self-model is built from traces plus a random audit stream and reports unmeasured capabilities as such (Proposition 5).

B. Request lifecycle

Fig. 1 shows one request and Fig. 2 its state automaton. The context manager assembles the prompt from the conversation, any retrieved memory and, when a knowledge base is selected, its passages, each with provenance. System 1 produces a draft in non-thinking mode. The gate computes features: mean and minimum token log-probability of the draft, the lowest window of consecutive tokens, agreement among k cheap samples, the domain class, whether the turn repeats the previous one, and, for verifiable tasks, the verifier's verdict on the draft. A verifiable task whose draft fails verification is never eligible for the keep branch; before it is escalated, four cheap System 1 samples are drawn in one request and the first that passes the exact check is returned (Section VII). On escalation, System 2 runs draft-blind in thinking mode with budget b and sequential sampling: each candidate is verified and the first accepted one is returned; a rejected candidate's evidence (failing test, counterexample, schema error) is fed into the next round, up to R rounds. If the budget is exhausted the reply is an explicit abstention with the best candidate and an uncertainty notice. Every reply carries an *assurance level*: `verified` (an exact verifier passed), `consensus` (soft agreement only), `unverified` (a kept draft on a non-verifiable task) or `abstained`; the interface shows them as *Checked*, *Cross-checked*, *Not checked* and *Not sure*. The gate's decision record and the trace of the loop are written to the audit ledger.

C. Components

The system is one Rust workspace of thirteen crates and one static gateway binary (Fig. 3).

- **Gateway (`nafs-gateway`).** HTTP API with an OpenAI-compatible chat endpoint, SSE streaming of tokens and of controller events, WebSocket chat rooms, authentication, rate limits, Prometheus metrics and the embedded web interface (Fig. 4).

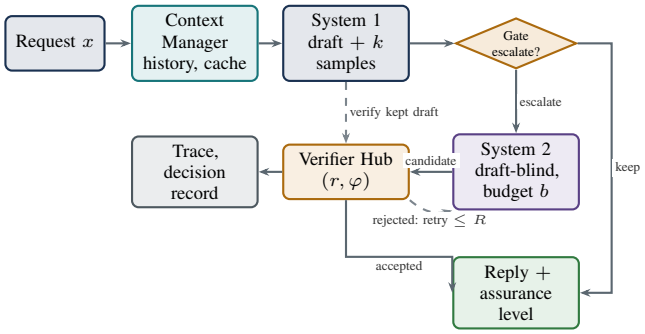
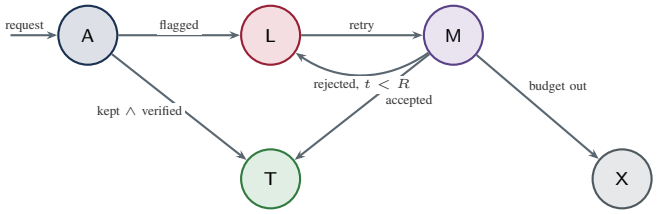


Fig. 1. Request lifecycle. Solid arrows are the main path; dashed arrows are the verification and retry edges.



A draft (fast path); L flagged by gate or verifier; M deliberate retry conditioned on verifier evidence; T accepted (assurance level attached); X abstain with uncertainty notice.

Fig. 2. Trace-state automaton of one request: a bounded retry loop (Lemma 1).

- **Orchestrator and gate (`nafs-core`).** The automaton, the gate, sequential verified sampling, budget forcing, timeouts and fallbacks.
- **Inference backends.** Two model servers behind one OpenAI-compatible contract. The default is our own engine, Smiley Core, served by `nafs-smiley-server`; `llama-server` and vLLM's CPU backend are alternatives switched at run time (Section VII).
- **Verifiers (`nafs-verify`).** Exact: a code sandbox (no network, CPU, memory and wall-clock limits; namespaces and seccomp on Linux, Seatbelt on macOS), numeric and symbolic mathematics, JSON Schema validation, and Lean 4. Soft: a rubric judge (the System 2 model in non-thinking mode at temperature 0) and sample consensus. Each publishes its measured (r, φ).
- **Agent harness (`nafs-harness`).** A bounded code agent whose runs end only when the exact verifier accepts or the budget is spent (Section VI).
- **Knowledge (`nafs-kb`) and memory (`nafs-memory`, `nafs-memorilla`).** Document readers, the passage index, the graphs, the check of an answer against its passages, the store of verified answers, the notes of earlier conversations and the memory-token module (Sections VIII and IX).
- **Audit ledger.** Per-domain calibration tables (escalation rate, calibration error of $\hat{e}_1$, verifier acceptance rates, latency quantiles) aggregated from traces and the audit stream, keyed by model hash; it feeds the quality page, drift alerts (a two-sided CUSUM per domain [84] with a shuffled negative control) and the gate's recalibration.
- **Evaluation and load testing (`nafs-eval`, `nafs-bench`).** Dataset loaders, paired statistical tests, a laboratory that runs each way of combining the two models as a *policy* on identical tasks, and a load generator.

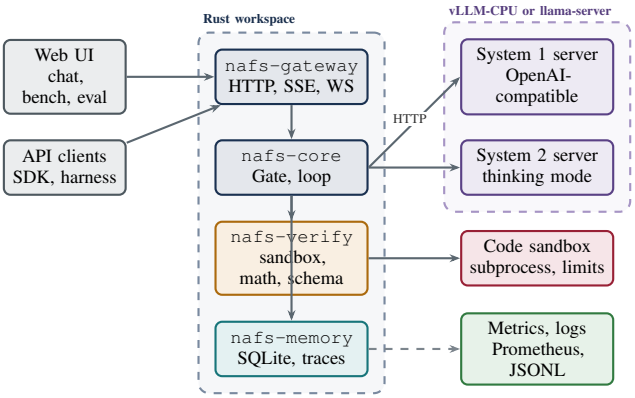


Fig. 3. System architecture, single node. The model servers are Smiley Core by default; llama-server and vLLM-CPU present the same API.

D. Model profiles

The reference profile uses two dense Qwen3 checkpoints [25], one per role: Qwen3-1.7B (Q4_K_M, 1.3 GB) as System 1 with thinking off, and Qwen3-4B-Thinking-2507 (Q4_K_M, 2.5 GB) as System 2. Dense grouped-query attention has the most mature CPU kernels and official GGUF files exist. The judge is the System 2 model in non-thinking mode, and embeddings come from Qwen3-Embedding-0.6B (Q8_0). All are Apache-2.0. Two alternatives are supported by configuration: a single switchable Qwen3-4B serving both roles, which halves resident memory at the price of a slower fast path, and a Qwen3.5 pair, whose System 1 candidate was measured and not adopted (Section X-H).

E. Deployment

The deployment is a single node: one gateway process, two model servers, an embedding server, a sandbox runner, SQLite for conversations and the ledger, and SurrealDB as the source of truth for the knowledge base. No request leaves the machine. The gateway switches between backend sets at run time, warm-first: every constant prompt prefix is primed on the new servers before one atomic swap, requests in flight finish on the pair they started on, and a new request finds a warm cache (about 1 s to the first token against 9.8 s cold). A GPU profile replaces the model servers with vLLM on CUDA and leaves every other component unchanged; it was not exercised here.

V. The Gate

The gate is the decision of Theorem 2: after the draft, escalate if and only if $L(\hat{e}_1 - \hat{e}_2) > \lambda \Delta c$, where λ is a price of latency in real units that the user sets with one "speed–care" control (Proposition 2). It adds no model call to a request: its features are by-products of the draft, and its own time is 0.08 to 0.12 s per escalated request.

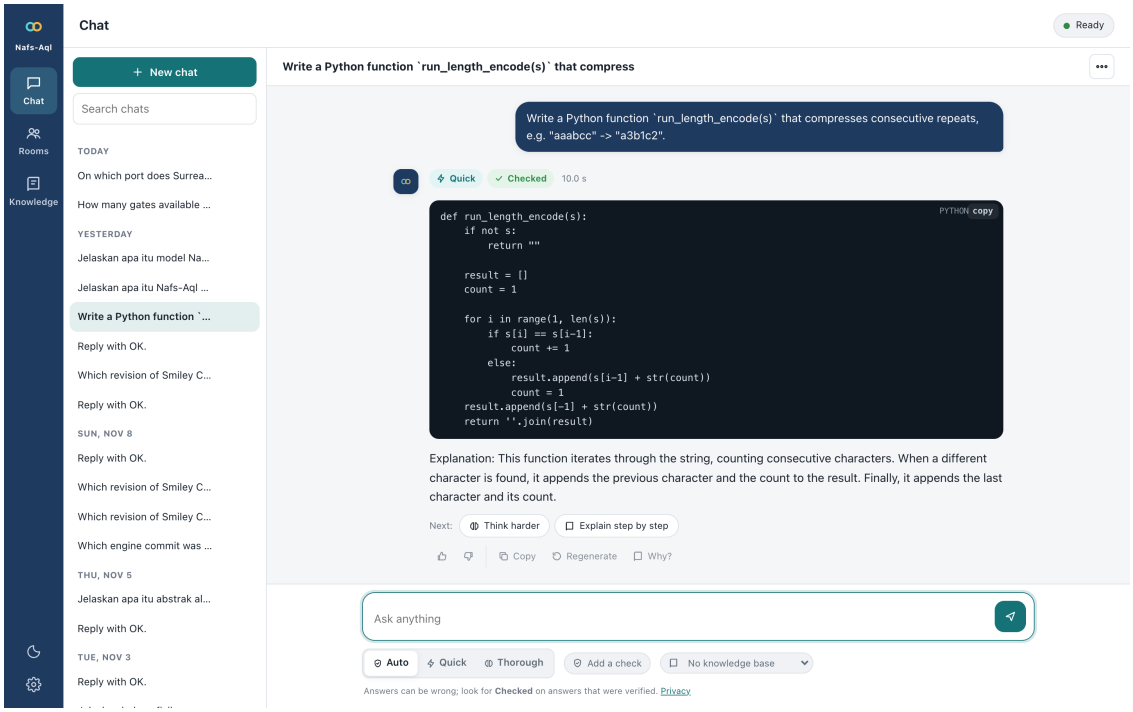


Fig. 4. The chat page. A code answer passed the exact verifier on the quick model's draft, so the gate kept it and the reply is labelled *Checked*.

A. Three rules, one decision record

Verifier first. On a verifiable task the exact verifier's verdict on the draft decides: a passing draft is kept as `verified`, a failing one is never kept. By Theorem 1 no confidence signal can add to an exact verdict, so the cheap samples are skipped there.

The fixed rule. Without a verdict and without a promoted calibration, the gate compares the draft's mean token log-probability with a threshold derived from λ (-0.175 at the default) and, inside an uncertainty band of 0.10 around it, the agreement of $k = 2$ cheap System 1 samples with the draft. Unverified mathematics drafts always get their samples, whatever their token confidence says, because their token confidence does not separate right from wrong (Section X-B). The samples are decoded in the draft's own batch and dropped if the draft is stopped early.

The calibrated rule. A fitted calibration maps the features (mean, minimum and lowest-window log-probability, agreement, domain, reask) to $\hat{e}_1$; $\hat{e}_2$ is a per-domain estimate from the audit stream. The calibration is fitted from the ledger's labelled drafts and is *promoted* to decide only when its held-out expected calibration error is at most 0.05 on enough samples (Proposition 1); until then the fixed rule decides and the calibrated estimate is only logged. A budget mode solves for the multiplier $\lambda^\star$ that meets a target escalation rate (Theorem 2(ii)).

Every decision is recorded with the rule that made it, the features, the estimates and the price, so a later refit can learn from it and a person can read why a request was or was not escalated.

B. Certified per-bin risk

A small calibration error bounds the *average* miscalibration. A stronger guarantee, taken from vLLM Semantic Router's cascade [49], bounds each score bin separately. The fitted calibration keeps B equal-mass bins of the held-out $\hat{e}_1$ (about one per hundred drafts, two to five) with their counts and, for each, the one-sided Clopper–Pearson [82] upper bound on its error rate at level α/B: the smallest u with $I_u(k{+}1,\, n{-}k) \geq 1 - \alpha/B$, found by bisection on the regularised incomplete beta function ($u = 1-(\alpha/B)^{1/n}$ when $k = 0$). A draft is kept only in a bin whose bound is at or below a configured risk; otherwise the decision says `uncertified_bin` with the bound, and a calibration with no certified bin is not promoted. Our exact verifiers supply the error counts at no cost. With $\alpha = 0.05$ and five bins, a bin of twenty drafts none of which was wrong still bounds its error at 0.21, so a young ledger certifies little and escalates more; the bound tightens with the ledger by itself. The rule applies to the calibrated mode only: budget mode fixes the escalation rate by design.

C. Unknown features and repeated questions

A draft without log-probabilities (a backend that does not return them, a failed sample) must not look like a confident draft. The gate applies a declared policy, `on_unknown` = `escalate` (default), `keep` or `fixed`, and the decision names the missing feature. A user turn whose content words overlap the previous user turn's at Jaccard ≥ 0.6 is a *reask*: asking again is evidence that the last answer failed. A reask of an unverified draft escalates under the fixed rule, and the feature is logged so a refit can learn its weight. The test is lexical rather than a cosine, so the gate runs without the embedding server.

D. Sequential evidence

Three mechanisms read a stream of evidence and stop on a bound.

Proposition 11 (Exact early escalation on a running minimum)**.** [exact] *Let $w(n)$ be the lowest mean log-probability over complete windows of W consecutive tokens among the first n tokens of a draft. Then w is non-increasing in n, so for the rule "escalate iff $w(N) < \theta$" the stopping time $T = \min\{n : w(n) < \theta\}$ gives exactly the full draft's decision, and no rule on the same stream escalates such a draft earlier without a non-zero disagreement rate.*

Proof. A minimum over a growing set; $w(N) \le w(T) < \theta$; before T every window is $\ge \theta$. □ □

A draft that the gate will reject anyway can therefore be stopped at the crossing and handed to System 2. On the mean-log-probability rule the same idea is only a heuristic (a draft whose running mean is below the threshold minus a margin is stopped), and Wald's bound shows it cannot be made more aggressive: with the measured per-token spread $\sigma \approx 0.28$ and margin 0.04, any sequential test at 5 % errors needs about 260 tokens, against drafts of 47 to 291.

Proposition 12 (Posterior stopping with a collapsing bound)**.** [exact for the rule; strong for optimality] *With k probed answers clustered into categories, counts v and a uniform Dirichlet prior, the posterior probability that the plurality answer is the majority answer is $1-I_{1/2}(v_{\text{top}}+1,\ k-v_{\text{top}}+1)$. Under a per-probe cost and a hard budget the Bayes-optimal rule stops at the first k with this probability at or above a bound $\theta(t_k)$ that is non-increasing in the budget used [17].*

Proposition 13 (Optimal probe interval)**.** [exact under an exponential model] *If the reasoning length at which the answer becomes available is exponential with mean m tokens, a probe costs c_p seconds and decoding runs at v tokens per second, the periodic interval minimising probe cost plus overshoot is $\Delta^\star = \sqrt{2c_p vm}$.*

Adaptive reasoning exit. The deliberate path is almost entirely System 2 reasoning, so the largest saving is to shorten the reasoning itself. System 2 reasons in chunks on a raw continuation endpoint, which keeps the prefix cached. At a reflection cue ("Wait", "Alternatively"), with a minimum gap of 64 tokens and at the latest every 256, the reasoning is closed provisionally and the model is asked for its answer in a few tokens (0.6 to 0.8 s). On a verifiable task the exact verifier judges that probe and, if it passes, the reasoning stops; on factual tasks two consecutive agreeing probes stop it, or optionally the posterior rule of Proposition 12. The final answer is then generated after the reasoning as on the standard path, so the exit changes only how much reasoning is generated. The gap of 64 tokens is within 1 % of the optimum of Proposition 13 at the measured values ($c_p \approx 0.7$ s, $v \approx 8$, $m \approx 300$ give $\Delta^\star \approx 58$).

E. Learning from its own traffic

The calibrated rule needs labelled drafts, and Proposition 5 says routed traffic alone gives biased ones. Two mechanisms supply labels without a System 2 call. A *probe* is a forced-fast draft that still draws its agreement samples and is judged by the exact verifier. A *label stream* runs probes on verifiable practice tasks while no user request is in flight; a user arriving cancels the draft in progress. At about 90 labels an hour the 2,900 held-out samples that a five-bin calibration estimate needs are about 32 hours of idle time. The quality page shows how many checked answers are held and whether the learned rule has been promoted.

TABLE II
THE LEAN PROTOCOL AGAINST REAL LEAN 4.35.0, FOR THE STATEMENT `A + B = B + A`.

Candidate	Verdict	Decided by
`by omega`	verified	Lean
`Nat.add_comm a b`	verified	Lean
helper lemma, then the theorem	verified	Lean
`by sorry`	partial (0.5)	Lean elaborates; `sorry`
`by rfl`	errors (0.05)	Lean error
`sorryAx _ false`	rejected	axioms report
`axiom cheat : False`	rejected	textual rule
shadowed `HAdd` instance, `rfl`	rejected	elaborated type

VI. VERIFICATION AND THE AGENT HARNESS

A. Exact verifiers

Four verifiers are exact on their domains. *Mathematics*: numeric equality with a reference and symbolic equality through SymPy. *Code*: hidden unit tests run in the sandbox, with per-assert credit protected by a per-run nonce so that a failing candidate still gets a graded score. *Structured output*: JSON Schema validation. *Proofs*: Lean 4 [85] under the frozen-statement protocol of Open Math Harness [55]. The statement is frozen on the service side; the candidate is refused on text alone for forbidden tokens (`sorry`, `axiom`, `native_decide`, commands that run code during elaboration) or for not reproducing the statement byte for byte; Lean then checks a file assembled from the frozen preamble, the candidate's body, an unguessable marker, the axioms report and the fully elaborated type, which must equal the pristine statement's character for character. Against real Lean 4.35.0 the protocol accepts three correct proofs and rejects every known attack at the layer assigned to it (Table II); the decisive case is a candidate that shadows the addition instance and closes the statement with `rfl`, which only the comparison of elaborated types rejects. A check takes about 0.6 s. Requests are refused when no toolchain is configured, so a Lean task can never be labelled verified by a fallback.

The soft verifiers are the rubric judge and sample consensus; both yield an assurance level below `verified`. Measured rates for all verifiers are in Section X-B.

B. Best-first repair

Every verified attempt of the deliberate path becomes a node of a tree, scored from the verifier's verdict alone (1 solved; for a partial result with C credited and L open steps, $0.5 + 0.4\,C/(C+L)$; 0.05 for errors; 0 for invalid). The next attempt repairs the node a policy chooses and sees that node's evidence and, as data, what earlier repairs of it did. With one candidate

explored at a time the policy is: highest score, ties to the fewest failures, then to the latest node; a node stays continuable until it has F children (default 2), which lets the search return to a parent when a repair made things worse.

Proposition 14 (Conservative extension)**.** [exact] *If every verdict is all-or-nothing (every failed candidate scores 0.05 and has one failure), the best-first policy repairs the latest candidate at every round, exactly as a plain retry loop does, for any fanout $F \geq 1$ along a chain of repairs.*

Proof. Along a chain each new node is a child of the previous one and has no children of its own, so it is eligible. All failed nodes tie on score and failures; the tie goes to the latest node. The root scores 0 and is chosen only when no eligible node scores more, which a valid failed candidate (0.05) prevents. □ □

The policy therefore changes behaviour only when per-assert credit, Lean's graded verdicts or an invalid patch distinguish candidates.

C. The code agent

A bounded agent works on a workspace of flat files. Each reply holds one action: a patch in Codex's `apply_patch` language [56], a whole file, or a `run` of a workspace file in the sandbox. The exact verifier runs after every change and a passing workspace ends the run at once; the model's own claim of completion never does. The best-scoring workspace is returned when nothing passes. Model calls are bounded by a step limit (default 6, at most 12) and the run by a wall-clock cap that holds inside a model call, as Lemma 1 requires of every loop.

Where a step spends its time. To first order

$$T_{\text{step}} = \frac{n_{\text{prompt}} - n_{\text{cached}}}{r_p} + \frac{n_{\text{dec}}}{r_d} + T_{\text{tools}}, \tag{5}$$

with prefill rate r_p and decode rate r_d. System 2 prefills about 31 tokens/s and decodes 6 to 8: a cold 1,969-token prompt took 64 s to prefill, and the same prompt with one more exchange appended took 3 s. Whether the cache is hit decides the cost of a step.

An append-only transcript.

Proposition 15 (Prefix reuse)**.** *Let a server reuse the longest common prefix between a request and the token sequence cached for the previous request of the run. If the files are part of the first message and are rewritten each step, then after a step that changed a file the reusable prefix ends at the first changed byte, and a run of K steps prefills $\Theta(K \cdot L)$ tokens for prompts of length L. If the transcript only grows, step k prefills only the previous reply and its observation, and a run prefills each transcript token once: $\Theta(L_K)$ for a final transcript of length L_K, plus one task message per compaction.*

The transcript is therefore never rewritten: a change of state is a new observation carrying the edited files or the verifier's evidence. Its size is metered from the server's reported prompt tokens plus an estimate of what was appended since. When it exceeds its budget, older observations are first cut to a head and a tail, and only if that is not enough is the transcript compacted into a *harness-written* state (the task, the current files, the last verdict and an "already tried" list). A model-written summary, as Codex uses, would cost minutes at these decode rates, and the facts it needs are already structured.

Actions under a grammar. On a backend that accepts a decoding grammar, each step's grammar admits only that step's legal actions: update and delete targets are the workspace's files, a hunk header cannot start with a line range, a `run` can name only an existing file, and the last step offers only edits. A reasoning model needs one more rule. A grammar applies from the first sampled token, and with a plain grammar Qwen3-4B-Thinking was forced to emit the answer in place of its reasoning; the grammar therefore opens with a free reasoning span up to the closing tag:

```
root  ::= think action
think ::= ( [^<] | "<" [^/] | ... )* "</think>" [ \n]*
action ::= patch | whole | run
whole ::= "```python\n" codeline* "```"
```

The reasoning stays unconstrained [60]. On a backend without grammars the harness reads free text with a lenient parser (a patch may start at its first file section, and unified-diff line ranges in a hunk header are dropped).

Which edits to offer. Let a whole file cost W answer tokens, a patch P, and let a patch fail to apply with probability f. The expected answer tokens of a repair are about $P + fW$, so a patch pays when

$$f < 1 - P/W. \tag{6}$$

Small models fail patch formats often, and a patch that applies can still break the file. The harness therefore offers the whole file while it is missing or at most 80 lines long, and a patch or the whole file above that.

Guards. (i) After an edit every changed Python file is compiled in the sandbox, never executed; a file that does not compile is not applied, and the model sees the error with the lines around it (37 ms per edit). (ii) A change judged below the best version so far is rolled back. (iii) A failed step's signature is the hidden tests' evidence, or the action and its first line of feedback; the second consecutive identical failure carries a nudge, the fourth ends the run as `stuck`. The same action text sent a third time draws a notice before it fails. After a step that reproduced an already-judged workspace the transcript is compacted and the next step is sampled hotter. (iv) An edit in a reply that ended on the token budget is refused. (v) A failed model call is retried twice with backoff. (vi) The first step of a System 2 run uses a 256-token reasoning budget and later steps 512.

A cascade. The gate's logic applies to the agent as well. The first step goes to System 1 as one request with four choices (the prompt prefilled once, the choices decoded in one batch at temperature 0.7); each candidate is applied to its own copy of the files and judged by the exact verifier; the first that passes ends the run. When none passes, the best-scoring candidate becomes the state and System 2 continues from step two.

VII. Inference on Four Cores

The reference machine is an Intel Core i3-8100: four cores, no simultaneous multithreading, AVX2 and FMA, no

TABLE III
BASELINE THROUGHPUT ON THE REFERENCE MACHINE (LLAMA.CPP, 4 THREADS, 512-TOKEN PREFILL, 128-TOKEN DECODE; MEAN ± S.D. OVER 3 RUNS)

Model (GGUF)	Size	Prefill tok/s	Decode tok/s
Qwen3-0.6B Q8_0	604 MiB	180.6 ± 2.4	32.6 ± 1.8
Qwen3-1.7B Q4_K_M	1.26 GiB	94.1 ± 9.5	15.5 ± 3.4
Qwen3-4B Q4_K_M	2.32 GiB	38.2 ± 0.6	6.7 ± 0.8
Qwen3-4B-Thinking-2507 Q4_K_M	2.32 GiB	38.4 ± 1.5	7.0 ± 0.9

AVX-512, 32 GB of dual-channel DDR4. A multi-threaded streaming read reaches 12.9, 17.2, 18.4 and 19.5 GB/s with one to four threads. By Proposition 8 decode speed is that bandwidth over the bytes read per token (1,094 MB for System 1, 2,503 MB for System 2), and prefill is bound by arithmetic. Table III gives the baseline throughput. Everything in this section follows from one fact: *decode is memory-bound and prefill is compute-bound*, so the gains are in not repeating work, not in more threads.

A. The engine

The default backend is Smiley Core, our own CPU inference engine: a Rust library with a GGUF loader, a byte-level BPE tokenizer, the Qwen forward passes with AVX2 and FMA kernels, an F16 or int8 key–value cache, grouped online-softmax attention, and telemetry that reports the bytes moved per token beside the tokens per second, so that a run can be checked against the memory roofline. A thin server exposes it through the same OpenAI-compatible contract as `llama-server`, with log-probabilities for the gate, top-k and min-p sampling, and a reasoning budget enforced by closing the reasoning span and continuing the turn. The engine's phase split puts 86 to 97 % of decode in the matrix products and the output head; sampling is 0.1 to 0.3 % and attention 1 to 3 % at these context lengths. It runs within 60 to 95 % of the machine's bandwidth.

Three threads to write, four to read. Three decode threads beat four in all six interleaved pairs (+4 % on System 1, +8 % on System 2 at the median): on a shared host the fourth worker is the one the scheduler preempts, and every matrix product waits for its slowest worker. Reading a prompt is the opposite case. It is bound by arithmetic, not by memory, and scales almost linearly with cores (22, 44, 60 and 72 prompt tokens per second on one to four threads for System 1). The engine therefore keeps two worker pools, a narrow one for decode steps and a wide one for prompt chunks; an idle pool's workers park, so the two never compete, and the weight products do not depend on how rows are shared among workers. With four threads for prompts and three for tokens, both models read prompts 23 % faster with unchanged decode speed (Section X-C). The same split exists in llama.cpp as separate thread counts for batch and generation.

A prefix store. A session with a single cache benefits only when a prompt follows a conversation with the same beginning; alternating a chat prompt and the agent's prompt prefilled the latter in full each time. The engine keeps the cache of each finished conversation as a snapshot (114 to 147 KB per position, 768 MB per model, least recently used evicted) and restores the one that shares the longest beginning with the new prompt, in the manner of prefix-sharing caches [68]. The answers are byte-identical with the store on and off.

TABLE IV
BATCHED DECODE, FOUR STREAMS AGAINST ONE (LLAMA.CPP SLOTS, IDLE MACHINE, TOKENS/S).

Model, context	1 stream	4 streams	gain	$c(4)$
System 1, short	15.8	36.9	2.34×	1.71
System 2, short	5.12	13.78	2.69×	1.60
System 2, 800 tokens	5.03	9.17	1.82×	2.24
System 2, 1,600 tokens	4.09	6.14	1.50×	3.40

Several users in one pass over the weights. A decode step reads every weight once, so serving n users one after another reads the model n times where one pass would do. Decode is bound by memory (19.5 GB/s) and not by arithmetic (about 460 GFLOP/s), and Q4_K needs 0.56 bytes per weight, so the roofline ridge sits near seven rows: up to about seven users, another row costs almost no extra time. The forward pass takes a position and a cache *per row*; rows of different users share one read of the weights while each attends only to its own history, one row at a time, so a user's logits are bit-identical to a solo run's. A slot scheduler runs either one chunk of one prompt or one token for every ready user in a step, in the manner of vLLM's serving core [67], and the prefix store is shared by all slots.

A kernel guard. Updating the engine once produced a fourfold decode slowdown with byte-identical kernels: a helper built with `std::array::from_fn` was inlined or not depending on how much unrelated code the crate contained, which put a function call inside the dot kernel (679 ns per row against 96). A kernel whose speed depends on an optimiser decision about code elsewhere has to be guarded by a benchmark; one is now part of the crate.

B. Batching and context depth

A server that decodes its slots in one batch makes a few more streams cheap, until compute saturates. Let $c(N)$ be the cost of a batch of N streams in units of one stream. Table IV shows that the gain falls with context: from 2.7× to 1.5× for System 2, and a repair prompt (the earlier candidate and the verifier's evidence) is about 1,600 tokens deep.

One request with n choices. Candidates sent as separate requests each prefill the whole prompt, and prefill gains nothing from batching. With a 703-token prompt and a 24-token answer one request takes 31.7 s, four separate concurrent requests 124.5 s and one request with $n = 4$ 39.1 s: the server prefills once and forks the cache. Every multi-candidate round is sent this way, and each choice is verified as its own stream ends.

When does a concurrent round pay? Take $R + 1$ rounds, one-attempt success probability p and independent candidates. The verified answers per single-attempt time are

$$U(N) = \frac{1 - (1-p)^{N(R+1)}}{c(N) \sum_{k=0}^{R} (1-p)^{Nk}}. \tag{7}$$

Proposition 16 (Limits of widening)**.** [exact] *As $p \to 0$, $U(N)/U(1) \to N/c(N)$: widening pays iff the batch is*

cheaper than the same streams in series. As $p \to 1$*,* $U(N)/U(1) \to 1/c(N) < 1$*: it never pays.*

Proof. For small p, $1-(1-p)^{N(R+1)} \approx N(R+1)p$ and the sum tends to $R+1$, so $U(N) \approx Np/c(N)$ and $U(1) \approx p$. For $p \to 1$ numerator and sum tend to 1. □ □

With the short-context costs the policy widens for $p \leq 0.48$; with the 1,600-token costs it widens only for $p \leq 0.07$. Concurrent hedging on System 2 is implemented and off (Section X-H).

Cheap samples before the expensive model. Where batching is cheap (System 1, short context) and the verifier is exact, the cores are better spent *before* System 2. When a draft fails its exact check, four System 1 samples are drawn in one request, each verified as it finishes, and the first that passes is kept.

Proposition 17 (Rescue break-even)**.** [exact] *Let* f *be the share of requests whose draft fails,* r *the share of those that a sample rescues,* ρ *the cost of the rescue stage and* E *the cost of an escalation. The saving per request is* $f\,(rE-\rho)$*, positive iff* $r > \rho/E$*.*

The agreement samples of the gate are fused into the draft's batch for the same reason: drawn after the draft they cost $1+c(2) = 2.33$ draft times, in its batch $c(3) = 1.78$.

C. The model switch

Both models are resident, each in its own process with its own mapped weights, so a switch inside a request is routing plus a warm prefix: there is nothing to freeze, copy or reload. From the gate's decision to System 2's first reasoning token takes 0.4 to 1.4 s (median 0.85 s over six escalated requests). When both servers decode at once each keeps about half to two thirds of its solo rate and the pair writes 7.1 to 7.8 tokens/s together under every thread and polling setting tried; the cores are the bottleneck and no co-scheduling policy was worth building. At the reasoning budget the server continues the turn with the reasoning so far as the assistant's prefix; the prefix must repeat the generated text exactly, or every reasoning token is prefilled again (5 tokens and 0.2 s when exact, against 108 tokens and 2.2 s at a budget of 200 when one newline differed).

D. Alternative backends

`llama-server` keeps four things the default engine's server does not expose: decoding grammars, n-choice requests, a raw continuation endpoint and four parallel slots per model. With Smiley Core as the backend the rescue and the cascade therefore have one candidate instead of four, the agent relies on the free-text parser, and the adaptive reasoning exit is off; llama.cpp is one command away and is the choice for a workload dominated by the code agent. vLLM's CPU backend runs the same API in a Linux container; on an AVX2-only host it must run in FP32 with a small context and is a portability profile, not a speed contest. Ollama is not supported because its OpenAI endpoint does not return log-probabilities, which the gate requires.

VIII. KNOWLEDGE

A person uploads documents and the assistant answers from them, with each claim pointing at the passage that states it (Fig. 5).

A. Reading and storing

Readers written in Rust handle PDF, Word, PowerPoint, LaTeX, HTML, Markdown and plain text. A heading-aware chunker cuts passages of 1,200 characters with 150 of overlap, each carrying its heading path and page.

Reading a PDF page. A PDF holds positioned runs of text, not paragraphs. A layout pass, our Rust rewrite of LiteParse's rules [74] over the reader's runs (box, size, weight, style), turns them into Markdown in reading order. Columns are found by recursive XY-cut: a gutter is a valley in the character density of a region's narrow lines, and the lines that cross it (a title, an abstract, a wide caption) are cut off first as bands, so the columns are read one after the other. A region whose rows split into three or more aligned cells is a table and is not cut into columns; its rows are written as Markdown rows, so a row stays one line. Running headers and page numbers are removed by position and repetition; paragraphs are rejoined across line-end hyphens, column breaks and footnotes; headings come from sizes above the body size, from numbering ("I.", "A.", "2.5") in bold, italic or capitals, and from short bold blocks. Table-of-contents entries are not headings. If the pass fails on a document, the reader's own converter reads it. On 25 papers (421 pages) parsing and chunking take 15.5 ms a page. SurrealDB is the source of truth for documents, passages, vectors and graphs. Passages are embedded by Qwen3-Embedding-0.6B, served by the same engine with a pooling head; a re-read keeps the vector of every passage whose embedded text did not change, and the embedder keeps the cache rows of the tokens a text shares with the previous one (every question starts with the same instruction), which leaves the vectors identical. Ingestion runs on two lanes: parsing, chunking and storing on a fast lane that makes a document searchable by its words within seconds, and vectors and the model-built graph on a slow lane, one job at a time. Uploads are size-capped while streaming, archive budgets do not trust header sizes, queues, documents per base, passages per file and parser time are bounded, and deleting a base removes the originals, the passages and the passages copied into stored replies.

B. An in-process index

Search does not go to the store. The service holds, per base and per document, the passages, an inverted index over their words, their vectors and the terms they name, within a memory limit (512 MB by default; a base that does not fit keeps the store's own BM25 and HNSW indexes). A base is immutable once made and swapped on change, so a search in flight keeps a consistent view; a change touches one document's part; a generation counter keeps a load from installing rows read before a change; and every 30 s, off the search path, the index's counts are compared with the store's and the documents that differ are read again.

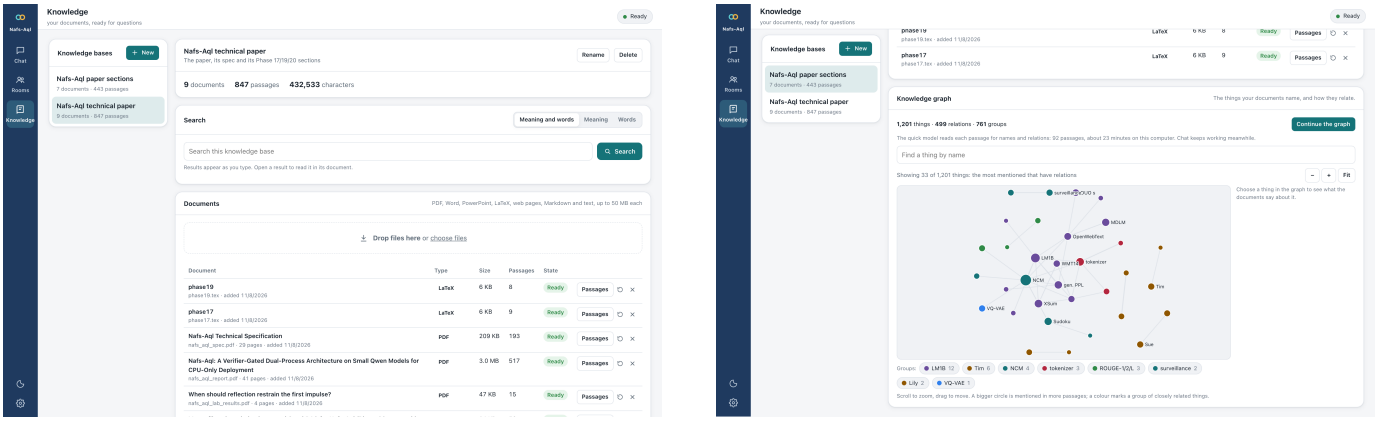


Fig. 5. The Knowledge page with this paper's own sources uploaded (left) and the graph read from them (right).

Words. BM25F over four fields (title, heading path, context line, body; $k_1 = 1.2$, $b = 0.5$) with lowercase, ASCII folding and Snowball stems, and one addition: an identifier is a term of its own beside its parts, so `vector_weight` or `Qwen3-1.7B` is not cut into words that match everywhere. Function words are left out of a query. The indexed text is the text a passage is embedded as, including a one-line context [73]. A question word of five letters or more that the base does not hold is taken for a typing slip and replaced by the nearest word the base does hold: same first letter, one edit away (two for a word of eight letters or more, an adjacent swap counting as one), the commonest of the nearest, at 0.7 of the weight of a word found as typed. Identifiers and numbers are never corrected.

Meaning. An exact scan of the normalised vectors. Below a few hundred thousand passages an exact scan is both the fastest and the most accurate index (100,000 vectors of 1,024 dimensions in 20.8 ms here). A base records the embedding model that wrote its first vector; if another model is running, the base is searched by its words and says so, until its meaning index is rebuilt, because vectors of two models are not comparable even at the same width.

Sentence windows. A passage's vector averages everything the passage says, so a question about one of its sentences can sit far from it. Each passage is therefore also cut into windows of about 240 characters (whole sentences; table rows and list items kept whole), each embedded with the passage's title and heading path. A passage's meaning score is the mean of its own similarity and the larger of that and its best window's, an idea taken from semtools' line-level search [75] but kept at the level of the passage, which is what the prompt receives.

Fusion. The two rankings are fused by reciprocal rank [72] ($k = 60$) with the meaning arm weighted 0.1. A passage whose lexical score reaches 10.0 is kept whatever its similarity; otherwise a passage must reach a minimum similarity to be shown to the model, so an off-topic question retrieves nothing rather than noise.

C. A graph with no model call

Terms are read from the text by rule: acronyms, identifiers, runs of capitalised words, and single capitalised words that are also capitalised away from the start of a sentence. A term named by at least two passages and at most a fifth of the base is a node, linked to the passages that name it with a weight that grows with its rarity. For an 847-passage base this gives 1,306 terms and 5,975 links in under a millisecond, so every base has a map the moment a file is read. Personalized PageRank over this bipartite graph (20 rounds of power iteration, seeded by the best passages of the search and the terms the question names) is available as a third ranking and serves the graph mode of chat.

D. A graph read by the model

On request, the quick model reads each passage once under a JSON Schema and returns entities, typed relations, a one-line context and the sentence that states each relation. Entities are merged by name and by the cosine of their name vectors (≥ 0.92), relations by their endpoints with a weight, and Louvain communities are detected. Three rules keep it faithful. Every word of an entity's name must occur in the passage. A relation is stored only with a sentence that occurs in its passage verbatim after whitespace normalisation, which also bounds what a document can put into a prompt through the graph. The entity types are data stored with the base (an ontology that a person can edit), and a stray type is mapped onto a declared one. In graph mode, passages cited by a fact join the candidate pool and the graph's ranking is fused with the search's as one more arm over the whole pool; a two-hop walk keeps the best eight relations per reached entity, scored by weight times 0.6 per hop, and shows the chain. Treating the graph as an arm rather than an override was the decisive change (Section X-E). Extraction costs 17 to 28 s per passage on this machine, so this layer is opt-in; the rule-built graph is always there.

E. Answering

Chat has the modes `rag` (numbered passages), `graph`, `memory`, `hybrid`, `auto` and `off`. Retrieved text is delimited and labelled as data, never instructions. A sentence packer fills one character budget with whole units: each sentence is scored by its own relevance to the question (idf-weighted overlap) plus its passage's rank, both normalised, and a unit

that does not fit is skipped rather than cut. Every reply names the searches that ran and why any did not, and a numbered citation opens its passage in context.

Follow-up questions. "And how fast is it?" names nothing a search can find. A question is read together with the previous user turn when it has fewer than three content words, or when it carries a reference word or an opening such as "what about" and at most six content words; a phrase that announces a new subject turns the rule off. The rule is a reduction of the topic-continuity signal of vLLM Semantic Router [49] to the one decision needed here, and uses no model.

Is the answer in its sources? An answer written from passages has no exact verifier, so it carries the lowest assurance level. It is nevertheless compared with what it was given, sentence by sentence and with no model. Citation marks, list numbers, questions and sentences that decline ("the passages do not say") are set aside. A remaining sentence is *supported* when at least 60 % of its stemmed content words are in one passage or repeated from the question, and every number it states occurs in the passages or in the question, with "1,200" equal to "1200" and "four" to "4". The reply lists the sentences that fail and why. The check says that a sentence is not in the sources; it does not say that the sentence is false, and it cannot see a wrong sentence built from the passages' own words. Systems that make this check with a token classifier of 150 to 300 M parameters [49] pay a model call per answer, which four cores cannot spare. Optionally, a quick answer whose supported share falls below a threshold is handed to the reasoning model with the same passages; this is off by default (Section X-E).

IX. Memory

A. A memory of verified answers

The context manager keeps conversations in SQLite and assembles each prompt within a context cap, with cache-affinity so that the engine's prefix store is hit. A retrieval memory holds earlier answers and offers them to later questions, under one rule: *only verified answers are stored*. A cache that keeps every successful response would, by Proposition 7, feed unverified text back as if it were knowledge. A hit is dropped when its stored question differs from the query by at most two content words with a negation cue or an antonym pair on one side only, so "is caching enabled" does not retrieve the stored answer to "is caching disabled" (a guard after vLLM Semantic Router [49]). Retrieved memory enters the prompt with its provenance and, like any retrieved text, as data.

B. A memory of earlier conversations

The second memory holds what was said, so that a fact told in one conversation can be found in a later one. It makes no model call.

Writing. Each finished turn of a stored conversation becomes a note (the question and the answer, reasoning removed, the answer clipped to 1,200 characters), and every third turn also a window over the last five. A greeting, a turn of under 30 characters and an answer that declined are not stored. A message that opens with "Remember that" is kept as a note that the size limit never removes.

Reading. Notes are read only when the question speaks in the first person or points back ("my", "last time", "you said"), and never when passages from a knowledge base are already in the prompt: on a CPU every thousand characters of prompt is seconds of reading, so an impersonal question pays nothing. Candidates come from BM25 over the notes and, when the embedding server runs, from the cosine of note vectors, fused by reciprocal rank. A note that answers the opposite question is dropped by the polarity guard of Section IX-A. The tail below the largest drop in relevance is cut. What remains is ordered by relevance, kind and age (a half-life of 30 days), cleared of duplicates, and cut to three notes and 1,500 characters. Two notes are duplicates when their word sets overlap by 0.9 *and* their numbers are bound to the same words, so "budget 10K" and "budget 12K" stay apart. The notes enter the prompt delimited as data.

Whose memory. A private request, an experiment and a chat room neither write nor read this memory; what is said in a room is shared by its members. Deleting a conversation deletes its notes. A person can read every note, delete one or all, add one, and merge repeated ones. At most 5,000 notes are kept; those never recalled go first, oldest first.

The extraction, the gate, the elbow and the number-aware duplicate test follow the conversation memory of vLLM Semantic Router [49]. The polarity guard, the exclusion of rooms and private chats, a size limit that uses the access counts, and the person's view of every note are added here; a model-written rewrite of the query is left out. A live check is in Section X-G.

C. The audit ledger as self-knowledge

The ledger is the system's memory of its own performance: gate decisions, verifier verdicts and audit outcomes per domain. It is what the calibrated gate is fitted from (Section V-E), and capabilities with no audited data are reported as unmeasured, never as zero (Proposition 5). Verified deliberate traces are also the raw material of distillation into the quick model, under the promotion rule of Proposition 7; training needs a GPU and was not run.

D. Memory tokens

Memorilla [77] compresses a document collection into a few vectors that a frozen decoder reads in place of the embeddings of placeholder tokens. Our port is a 31.5 M-parameter module (sixteen slots, two cross-attention blocks, eight heads, a projection from the embedder's width to the decoder's) written in Candle, with forward-pass parity to the unmodified PyTorch reference within 1.9×10^{-6} on three configurations. The engine admits requests whose placeholder positions carry supplied vectors, and a trainer fits the module to a knowledge base through a differentiable Qwen3 decoder (logits within 2×10^{-5} of a reference implementation). Document keys and values are prepared once per collection. The `memory` chat mode uses the module alone and `hybrid` adds it to retrieved

passages; answers read from memory tokens are decoded greedily, as the reference evaluates its own.

The module needs pre-training that a CPU cannot do, and the one module fine-tuned here *collapsed*: all sixteen memory vectors are one vector, whatever the question (Section X-F). Two measures follow. The trainer now uses the reference's from-scratch recipe (learning rate 10^{-4}, sixteen examples per step, five epochs, a warm-up from zero over a tenth of the steps, dropout 0.1 at the reference's four sites, a token-level loss). And the trainer computes at every validation the mean cosine between the slots of one question and between the memories of different questions, saves both beside the weights, and orders checkpoints by whether the memory depends on the question, then by the gain in claim accuracy over no memory, and only then by validation loss. A module whose memories for different questions have a cosine of 0.98 or more is not used: the automatic mode answers from passages, and an explicit request for memory falls back to passages and says why. A module saved without these statistics is probed once with three fixed questions. The modes stay off by default.

X. Evaluation

A. Setup and method

All measurements were taken on the reference machine of Section VII, which was shared with other workloads throughout (load average typically 5 to 30 on four cores, at times above 100). Absolute times are therefore pessimistic lower bounds on speed; token counts, verdicts and rankings do not depend on load. Three rules were applied to every comparison. (i) Arms are *interleaved* on the same items, with their order rotated, so that a drift in load hits every arm; twice a sequential run suggested a gain (30 % from a cache format, a recall gain from a graph arm) that an interleaved or re-based repeat removed. (ii) A change becomes a default only on a measured gain; otherwise it stays off and is reported. (iii) Sample sizes are stated; they are far below those of Proposition 6, so quality differences are indicative. Benchmarks are GSM8K [20] for mathematics, MBPP [86] for code, a JSON-Schema task set, and question sets over this paper's own sources for retrieval.

B. Does the gate buy accuracy at a lower cost?

Policies. Table V runs five ways of combining the two models on the same mixed tasks. With the verifier visible, the gate matched reasoning-only accuracy at 0.36 of its cost. With the verifier hidden, the uncalibrated gate routed correctly on only 50% of items, because cheap samples agreed with wrong drafts: agreement is a flag, not evidence (Theorem 6). Pooled over 6 stored runs, the policy with the lowest expected loss $L(1-\mathrm{acc})+\lambda c$ is, per domain: code: the gate with the verifier for $\lambda \le 0.1$, System 1 alone for $\lambda \ge 0.2$; math: the gate with the verifier for $\lambda \le 0.1$, System 1 alone for $\lambda \ge 0.2$; structured: the gate with the verifier at every λ.

Verifiers. Table VI gives measured true- and false-accept rates. Mathematics meets $\varphi \le 0.01$; the code φ is an upper bound inflated by behaviourally equivalent mutants. A judge on the quick model was measured as a cheaper alternative and rejected: System 2 judge: $r = 1.00$, $\varphi = 0.045$ (upper 0.23); System 1 judge: $r = 0.78$, $\varphi = 0.27$ (upper 0.50); 23 correct and 22 seeded wrong answers.

TABLE V
Policies on a mixed task set ($n = 6$ per policy; indicative)

System	n	Acc.	95% CI	Esc.	Wall (s)
fast_only	6	67%	[22%, 96%]	0%	13
nafs_aql	6	100%	[54%, 100%]	33%	96
nafs_blind	6	67%	[22%, 96%]	17%	133
slow_conditioned	6	100%	[54%, 100%]	100%	319
slow_only	6	100%	[54%, 100%]	100%	264

TABLE VI
Verifier quality: r from correct answers, φ from seeded mutants

Verifier	Correct	r	Mutants	φ	φ upper
code_tests	364	0.995	754	0.109	0.133
json_schema	24	1.000	53	0.000	0.067
judge	23	1.000	22	0.045	0.228
math_numeric	2000	1.000	2800	0.000	0.001

Which confidence signal predicts a wrong draft? 150 GSM8K items were drafted by System 1 under production settings and scored by the exact verifier (31 wrong). The best token signal is the single lowest token log-probability (AUROC 0.646 [0.539, 0.746]); the mean log-probability, on which the fixed rule rests, has 0.564 [0.449, 0.672], and at its production threshold it escalates none of the 150 drafts, because mathematics drafts have a mean of about -0.03 whether right or wrong. Agreement with two cheap samples is the stronger signal: on 60 of the drafts (12 wrong) its AUROC is 0.759 [0.618, 0.901], and 0.823 with ties broken by the lowest token; the rule "any disagreement" escalates 25 % of drafts and catches 8 of the 12 wrong ones (precision 0.53), which raises the expected cascade accuracy from 0.800 to 0.908–0.933. This is why unverified mathematics drafts always get their samples. The precision is still below the 0.65 that Theorem 2 requires at the default price, so a promoted calibration will price the signal rather than apply it unconditionally.

Certified bins on a live ledger. Probes on 204 GSM8K and MBPP training items gave 203 labelled drafts, 51 of them wrong. The fit is well calibrated (held-out ECE 0.024, Brier 0.117). Over 400 random 60/40 splits and about 1,300 held-out bin tests, one bin broke its bound significantly, so the certificate holds out of sample. At this size it costs coverage: on the same held-out drafts the fixed rule keeps 78.2 % at 10.1 % wrong, the calibrated rule on expected loss alone keeps 85.8 % at 16.6 % wrong, and with certified bins it keeps 63.6 % at 9.5 % (risk 0.40), 54.5 % at 9.9 % (0.25) and 5.4 % at 12.4 % (0.15). Bounds from a hundred drafts are too wide to beat a rule that uses the samples' agreement, so the fixed rule stays in force until the label stream has filled the ledger. The stream has since raised the ledger to 1,294 labelled drafts (280 wrong). A refit on them is as well calibrated (held-out ECE 0.023 over five folds, Brier 0.115), and of its five equal bins one is certified under the 0.15 bound (upper bound 0.106) and two sit just above it (0.157 and 0.161). The fit was not promoted: the

TABLE VII
LATENCY BY STAGE (SHARE OF WALL TIME; LLAMA.CPP BACKEND)

Path (replies)	Stage	Share	p50 (s)	p95 (s)
fast (475)	System 1 draft	98%	13.6	56.1
fast (475)	verify draft	2%	0.2	0.9
deliberate (39)	System 2 deliberation	95%	120.6	424.1
deliberate (39)	judge	3%	40.0	44.3
deliberate (39)	System 1 draft	2%	9.2	59.2
deliberate (39)	probe	1%	1.5	6.4

promotion rule asks for 2,900 held-out drafts, and the system applies its own rule.

Early escalation and the reasoning exit. In an agreement study, on 40 open-ended prompts, the configured rule ($N = 32$, margin 0.04) agreed with the full-draft decision on 100% of prompts, caught 3 of 10 escalations early and saved 116 draft tokens per early escalation. With the adaptive reasoning exit, with the exit: accuracy 100%, mean wall 123 s; without: 100%, 220 s ($n = 11$ GSM8K items; paired Δ accuracy +0 points, 95% CI [+0, +0]). On six factual prompts the posterior exit rule gave the same content as the two-agreeing-probes rule with slightly more reasoning (382 against 357 tokens on average), so the simpler rule is the default.

C. Where does the time go, and what speeds it up?

Over 514 replies with stage timings (Table VII), the fast path spends essentially all of its time in the System 1 draft and the deliberate path in System 2 deliberation; verification and gating are negligible. Decode speed is the bottleneck, and it falls several-fold when other workloads share the cores.

The engine. With identical greedy requests (12 interleaved pairs per model) Smiley Core decoded 14.3 tokens/s against 12.6 for llama.cpp on System 1 and 7.4 against 6.6 on System 2, faster in 10 and 11 of 12 pairs; answers were identical in 12 of 12 and 9 of 12 pairs. Through the whole gateway, on twelve prompts in automatic mode with the backend switched before every request, both backends checked 9 of 9 checkable answers; the median request took 9.7 s against 17.9 s and the twelve 131 s against 303 s (faster in 10 of 12 pairs, one-sided sign test $p = 0.02$), while the median first token was later (1.50 s against 0.97 s).

Not repeating work. Table VIII collects the measured effect of each mechanism of Section VII.

Several users. Table IX gives decode throughput with each engine alone on the machine. At two users Smiley Core is $1.40\times$ ahead; at four llama.cpp is ahead ($1.30\times$). Every user's text from Smiley Core was identical to a solo run at every concurrency (4 of 4 at four users); llama.cpp's was not (0 of 4). The gateway does not yet use this concurrency for n-choice requests.

Load. On the llama.cpp backend the fast path at 1, 2 and 4 concurrent sessions had a p95 latency of 7.2, 4.9 and 9.0 s; the ratio of the four-session to the single-session p95 ranged from 1.25 to 2.48 across runs on the shared host.

TABLE VIII
MECHANISMS THAT PAY, WITH THE MEASUREMENT BEHIND EACH DEFAULT.

Mechanism	Measured effect
Skip cheap samples when an exact verifier decides	median draft time 13.4 s to 7.9 s
Prompt-cache warm-up at start	first token 9.8 s cold, about 1 s warm
Agreement samples fused into the draft's batch	time to the gate's decision 23.1 s to 17.9 s ($-24\,\%$; faster in 23 of 28 pairs, $p = 0.0005$)
n choices in one request	39.1 s against 124.5 s for four requests
Rescue: four System 1 samples after a failed check	3 escalations avoided in 40 paired items; break-even 106 s against 250–580 s per escalation
Three decode threads	$+4\,\%$ (System 1), $+8\,\%$ (System 2), 6 of 6 pairs
Four threads for prompts, three for tokens	prompt reading $+23\,\%$ on both models (49.7 to 61.0 and 18.5 to 22.7 tokens/s), time to the last token $-16\,\%$; 7 of 8 and 6 of 6 pairs
Prefix store	switching the kind of request: 5.8 s and 14 s of prefill to 0.1 s and 0.2 s
Exact continuation prefix	escalated requests 182 s to 133 s on average (not interleaved; prefill counts are deterministic)
Adaptive reasoning exit	220 s to 123 s at equal accuracy ($n = 11$)
Embedder keeps a shared prefix	question embedding 297 ms to 146 ms, identical vectors
Embedding server on four threads	a question in 252 ms against 293 ms, a passage in 1.08 s against 1.27 s; 24 of 24 pairs each

TABLE IX
DECODE THROUGHPUT OF QWEN3-1.7B, ALL USERS TOGETHER (TOKENS/S).

Users	1	2	4
Smiley Core	14.2	20.2	24.9
llama.cpp (slots)	14.4	14.4	32.4

D. Does the agent harness verify more, sooner?

Ten small Python tasks (fixes of a buggy function and functions written from an empty workspace, four or five hidden asserts each) were run through the gateway once per arm with a budget of six steps (Table X).

The append-only transcript with a per-step grammar verified more tasks on both models, and the mechanism is visible in every step: 23 of 50 steps of the free-text loop held no action the harness could parse, against 1 of 41 under the grammar. The difference in verified tasks on System 1 rests on two discordant tasks (exact McNemar $p = 0.5$). Offering a patch at every step was the worst arm: most patches applied and still broke the file, so the grammar fixes the syntax of a patch, not its sense. A verified System 2 fix of a small function takes 33 to 59 s with the cheap first step, where the free-text loop spent single calls of 500 to 1,041 s and verified none. The cascade verified the same ten tasks in 26 % less wall time (faster in 7 of 10 pairs, sign test $p = 0.17$); six tasks never reached System 2. The guards (compile check, rollback, stall guard) cost nothing: with them on and off, ten tasks on each model verified identically (10/10 on System 2; 4/10 and 5/10 in two rounds on System 1), with the same number of model calls. They also changed no outcome on this task set; they are insurance against failure modes seen in practice, such as a run that spent its whole 900 s cap on repeated indentation errors. Pruning old observations before compaction is likewise inert here: the model is never shown more than about 930 bytes of evidence, and 16 paired runs had identical action sequences.

Lean. The protocol is validated end to end with the real

TABLE X
CODE AGENT, INTERLEAVED LIVE COMPARISONS ON THE REFERENCE MACHINE. SAME TASKS, MODEL AND STEP BUDGET PER ARM; ARM ORDER ROTATED PER TASK. MALFORMED: STEPS WHOSE REPLY HELD NO ACTION OR A PATCH THAT DID NOT APPLY. WALL TIMES ARE FROM A SHARED HOST.

Comparison	Arm	Verified	Calls	Malformed	Decoded tok.	Wall (s)
System 1, transcript and grammar	context rebuilt each step, free text	2/10	50	23/50	2,793	356
	append-only, grammar, edits by file length	4/10	41	1/41	4,146	504
	append-only, grammar, patch always offered	0/10	60	16/60	10,706	1,468
System 2, same (3 tasks)	context rebuilt each step, free text	0/3	15	13/15	9,907	2,695
	append-only, grammar	3/3	3	0/3	1,818	451
System 1, breaking repetition	without	5/10	35	0/35	3,926	523
	with	7/10	27	0/27	2,575	305
System 2, reasoning budget (6 tasks)	512 tokens every step	6/6	6	0/6	3,499	560
	256 on the first step, then 512	6/6	6	0/6	1,961	282
Cascade, System 2 as the run's model	System 2 from the first step	10/10	12	0	4,534	1,987
	four System 1 candidates first	10/10	14	0	5,958	1,467

TABLE XI
SEARCH TIME, 40 QUESTIONS INTERLEAVED THROUGH BOTH PATHS, 847 PASSAGES (LOAD 6 TO 8). UNDER HEAVIER LOAD (24 TO 129) THE STORE PATH TOOK 5.3 S FOR A HYBRID SEARCH AT THE MEDIAN.

	Store	In-process index
Search by words, median (p90)	177 ms (221)	0.086 ms (0.100)
Hybrid search, median (p90)	578 ms (649)	152 ms (181)
Hybrid without the embedding call	–	0.17 ms
The same with the PageRank arm	–	0.60 ms
Graph of a base	17–28 s/passage (model)	under 1 ms (rules)

TABLE XII
WHERE THE ANSWERING PASSAGE RANKS, 256 QUESTIONS. POOLED MEAN RECIPROCAL RANK: 0.379 THROUGH THE STORE, 0.388 THROUGH THE INDEX.

Set	Path	R@1	R@8	MRR	both@8
Quoted (60)	store, hybrid	0.767	1.000	0.855	
	index, hybrid	0.817	1.000	0.882	
Reworded (146)	store, hybrid	0.096	0.226	0.147	
	index, hybrid	0.116	0.240	0.160	
	index, vectors alone	0.062	0.137	0.088	
Two-passage (50)	store, hybrid	0.300	0.800	0.485	0.62
	index, hybrid	0.260	0.830	0.460	0.68

binary, but Qwen3-4B-Thinking in 4 bits does not yet write valid Lean 4: for $n + 0 = n$, which `rfl` proves, it abstained after three attempts (826 s). Every verdict was precise and no wrong proof was accepted; the system abstained with a notice.

E. Does the knowledge base find and use the right passage?

Answers. Twenty questions with exact expected phrases were asked over this paper's own sources in each mode, arms interleaved per question. On a 66-passage base with the model-built graph (271 entities, 100 relations, each with a verified quote): closed-book 0/20, `rag` 18/20 (median 20.8 s), `graph` 17/20 (25.2 s). On the 847-passage base: closed-book 0/20, `rag` 17/20 ($p = 1.5 \times 10^{-5}$ against closed-book). On an earlier fixture base closed-book answered 2/20 against 18/20 for `rag`. Before the graph became a fused arm it had answered 15/20 and lost all three discordant pairs to `rag`, because it reordered passages by entity mentions; as an arm it loses one pair and wins none. It no longer costs accuracy, adds none on these questions and costs 15 % of the time, so the automatic mode uses `rag`. Every miss is classified: two questions were not retrieved, one was retrieved but not used.

Speed. Table XI compares the in-process index with searching through the store.

Ranking. Three question sets over the 847-passage base, each accepted phrase checked by script: 60 that quote their passage, 146 reworded ones (on average 9 % of a question's content words occur in the answering passage) and 50 that need two passages (Table XII). The two paths are equally accurate within what the sets resolve. Fifty-nine settings were ranked on these sets; the only change adopted was length normalisation $b = 0.5$ (the answer first for 53 of 60 quoted questions against 50 at 0.75). The weight of 0.1 on the meaning arm was confirmed: with equal weights the fused list is worse than words alone. Passages of 1,200 characters tie for first on recall and are best on rank against 600 and 2,400.

Reading and windows. The same nine files were stored twice through the upload path, once read by the PDF converter used before and once by the layout pass, and the windows were then added to the second base. Table XIII scores only the questions whose accepted phrase occurs in both bases (56, 142 and 44), because the phrases were written against the earlier reading. Against the LaTeX sources of five of the project's own documents, the layout pass marks 166 of 173 section titles as headings against 11, holds 73.5 % of the sources' five-word runs in order against 69.2 %, and adds fewer runs found nowhere in the source (47.0 % against 50.7 %). On 25 papers it finds 833 headings against 195, leaves 19 passages without a heading path against 32, and reads a page in 15.5 ms against 24.0. Better passages helped most where the question shares few words with the answer: a reworded question finds its passage first half as often again, and a two-passage question a third as often again. The windows then lifted quoted questions most (first place 0.786 to 0.875), the meaning arm alone most of all (quoted MRR 0.465 to 0.611), and made no set worse; they cost about one more embedding pass per document (1,402 windows for 723 passages). Weighting the meaning arm more than 0.1 again helped reworded questions and cost quoted ones,

TABLE XIII
WHERE THE ANSWERING PASSAGE RANKS WITH THE SERVICE'S HYBRID SEARCH, THE SAME NINE FILES READ TWO WAYS, QUESTIONS ANSWERABLE IN BOTH BASES.

Set	Base	R@1	R@8	MRR
Quoted (56)	converter, 847 passages	0.768	1.000	0.860
	layout, 723 passages	0.786	0.982	0.870
	layout + windows	0.875	0.982	0.922
Reworded (142)	converter	0.099	0.232	0.152
	layout	0.148	0.246	0.194
	layout + windows	0.162	0.239	0.201
Two-passage (44)	converter	0.284	0.830	0.474
	layout	0.375	0.909	0.570
	layout + windows	0.386	0.920	0.571

TABLE XIV
QUERY-SIDE RULES ON THE 847-PASSAGE BASE, HYBRID SEARCH.

	R@1	R@4	R@8	MRR
Mistyped (60), as typed	0.750	0.950	0.967	0.831
Mistyped (60), corrected	0.750	0.983	1.000	0.844
Follow-up (40), alone	0.025	0.125	0.225	0.073
Follow-up, with the previous turn by rule	0.250	0.325	0.375	0.285
Follow-up, always with the previous turn	0.275	0.350	0.400	0.310

so the weight is unchanged.

The open problem is still the reworded question: the answering passage is first for 16 % of such questions and within the first eight for 24 %.

Typing slips and follow-ups. Table XIV measures the two query-side rules of Section VIII against searching the question as typed. For slips, one word of each quoted question was corrupted by a seeded swap, deletion or replacement (56 of 60 questions changed). The correction repaired 40 words, brought the two questions that had lost their passage back into the first eight, and changed no word of the 60 questions as written. For follow-ups, 40 pairs were written from facts in the sources: a question that names its subject, then one that refers back ("How was that fixed?"). Searched alone, a follow-up finds its passage first once in 40; read with the previous turn, ten times in 40. The rule fired on 37 of the 40 follow-ups and on 1 of the 256 self-contained questions of the three ranking sets.

Is the answer in its sources? The support check of Section VIII-E was measured on 50 questions with exact expected phrases. The quick model answered each twice, from the passages (42 of 50 right) and with none (0 of 50), and the answers made up without passages were checked against the passages the other answer had been given (Table XV). A made-up answer is flagged 38 times in 44; a right answer from passages once in 42; the area under the curve for telling the two apart is 0.90. The check does not catch a wrong reading of the right passage: of the five wrong answers from passages that make a claim, three quote a number that is in a passage and is the wrong one. Two design points came from this run. A word that the answer repeats from the question counts as found (without it five right answers had a sentence flagged, such as "listens on port 8300" against a passage that writes `:8300`); requiring instead that every other word be in a passage flagged 16. And a sentence that says the base does not hold the answer is set aside rather than counted as an unsupported claim. Sending answers below a supported share of 0.5 to the reasoning model would have moved two of the 50, both wrong; two answers do not show that the reasoning model would then be right, so that rule is off.

TABLE XV
THE SUPPORT CHECK ON 50 QUESTIONS, QUICK MODEL, 847-PASSAGE BASE. AN ANSWER IS FLAGGED WHEN AT LEAST ONE OF ITS SENTENCES IS NOT IN THE PASSAGES.

Answers	With a claim	Flagged	Mean share
From passages, right (42)	42	1	0.99
From passages, wrong (8)	5	2	0.60
Made up, no passages (50)	44	38	0.21

Scale and readers. A synthetic base of 1,000 documents and 50,000 passages was timed through the store under load 20 to 130: listing the documents 0.28 s, a search by meaning through the HNSW index 0.37 s at the median (0.93 s at p95); an exact in-memory scan of the same vectors takes 18 to 22 ms. Deleting such a base runs in the background at about 16 passages a second. As a check of the readers, this paper was read from its LaTeX sources and from its PDF: 98.0 % of the numbers and 91.6 % of the distinct words of the former are in the latter; the remainder are macro names that the LaTeX reader lets through inside mathematics.

F. Do memory tokens carry knowledge?

Not at this scale. The module was fine-tuned for one epoch on the 66-passage base (234 examples; validation loss 8.77 to 1.46). On the 20 questions: closed-book 0/20, `rag` 18/20, `hybrid` 17/20, `memory` alone 0/20 with degenerate text. In the training decoder the claim accuracy is 0.55 with the module and 0.55 without it: the module read no knowledge, as its authors predict for a module without pre-training. Training costs 137 to 272 s per example here, and the pre-training stages need a GPU. An untrained module reads exactly like no module, so the path itself is correct.

A second look at the trained module explains the result. Its forward pass, recomputed outside the system and matched to the reference within 9×10^{-7}, gives *one vector for all sixteen slots and for every question*: the cosine between slots is 0.9999999, between the memories of different questions 0.999998, and the vector's norm is 63 where a token embedding of the decoder has 1.5. The residual branches had drifted to large constants and the projection was close to rank one. The module had become a fixed prompt for the answer format, which lowers the loss and carries no knowledge, and selecting the checkpoint by validation loss had rewarded it. Against the reference, the first trainer used three times the learning rate at a quarter of the batch, no dropout and a warm-up that started near the peak. These are corrected and the collapse is now measured and refused (Section IX-D); a retrain with the corrected recipe was not run, since it costs hours for one small base and its outcome without pre-training is uncertain.

G. Does the memory of earlier conversations work?

A live check of fourteen assertions passes. A fact kept on purpose ("Remember that the staging server listens on port

8443") and a fact stated in passing in another conversation were each recalled from a new conversation and used in the answer; a question in other words ("which pet does my sibling own?" for "my sister's dog is called Biscuit") found its note through the meaning arm; an impersonal question and a private request read nothing; the conversation that held the fact answered from its own history; a forgotten note and the notes of a deleted conversation were not recalled; and deleting the conversations left the memory as it had been. When the gate opens, the memory step takes 0.13 to 0.58 s under load 20 to 54, nearly all of it one embedding call; when it does not, the step costs nothing. This is a check of function, not of recall quality over a long history: no set of real multi-session conversations with known answers was available.

H. What did not pay

Table XVI lists what was built or measured and not adopted. Each stays available as an option where it could pay on other hardware.

I. Validation

The workspace has 513 Rust tests, including the ported upstream suites of the patch parser and the Lean rules, bit-identity tests of the batched engine and parity tests of the memory module; formatting, lints and dependency audits are clean. Live sweeps on the final build pass: the API contract, 26 of 26 security probes (sandbox escape, injection, authentication, limits) with no open high-severity finding, 33 of 33 knowledge-security checks, 69 of 69 end-to-end checks of every endpoint and page, 10 of 10 chat-room checks, 55 knowledge checks, 14 of 14 checks of the conversation memory, 53 of 53 checks of the agent harness and all nine acceptance tests (the first token of a quick reply after 0.25 s; a verified deliberate answer after 155 s against a target of 240 s). The acceptance latencies depend on what else the machine is doing: an earlier sweep the same day, under a load average of 30 to 40 from other workloads, missed both targets (2.6 s and 420 s) with correct answers. The security suite earlier found one high-severity issue, since fixed: sandboxed code could signal other processes.

XI. Comparison with Related Systems

Five open projects were read in full against this system, and mechanisms were taken from each where they cost no model call. This section says what each is, where the designs differ, and what was measured side by side.

Codex and Open Math Harness. Codex [56] is a coding agent whose turn loop assumes a frontier model with native tool calls; Open Math Harness [55] is a capability microkernel for mathematics research. Neither was embedded. From Codex we took the patch language with its lenient parser and fuzzy matcher, the append-only history, grammar-constrained patches, truncation on insertion, budget reminders and retry backoff; from Open Math Harness the Lean protocol, the scoring of partial results and the candidate tree. What changed for small models on a CPU is in Section VI: compaction is written by the harness rather than by the model, the grammar is rebuilt per step and leaves the reasoning free, and the edit format is chosen by file length.

vLLM Semantic Router [49] is a decision layer in front of many models: about twenty signal types, Boolean rules over them that pick a decision, and a selection algorithm inside it. Its decision rule is lexicographic and hand-weighted; ours is one rule derived from expected loss with a price of latency in real units. It decides correctness by model confidence, self-rating or peer agreement; ours by exact verifiers. Its cascade serves the last response when none passes a threshold; ours returns the best-scoring candidate or abstains. Its semantic cache stores every successful response; our memory stores verified answers only. It reports its own gate at 79 ms to 9.8 s per request on a CPU, because the signals come from a model of 0.3B parameters or more; ours costs about 0.1 s from features the draft already produced. From it we took certified per-bin risk, a declared policy for unknown features, the reask signal and the polarity guard; and from its retrieval and memory code the model-free parts of a conversation memory (the notes, the gate that decides when to read them, the elbow threshold, duplicates that respect numbers) and the idea of a topic-continuity signal, reduced to one rule for follow-up questions. Its document search re-ranks the vector hits with BM25 and character trigrams computed over those hits only; ours runs words and meaning as independent arms over the whole base, which is what finds a passage the embedder missed. Its check of an answer against its context is a token classifier of 150 to 300 M parameters; ours is a rule over words and numbers (Section VIII-E). Its selection algorithms choose among many models (Elo, nearest labelled queries, Thompson sampling, session-aware switching); with two models there is nothing to select, and the one question-only signal we could test did not add to the draft's own (Table XVI). Its repository commits speed records and no quality measurement for retrieval, memory, cache precision or selection. It remains ahead on breadth: Envoy and Kubernetes integration, trained jailbreak and personal-data classifiers, per-user memory isolation, a management dashboard. Those need a classifier model per request or a service mesh, and are outside what a single-binary assistant on four cores should carry.

Memorilla [77] trains a small module that reads a document collection into sixteen vectors for a frozen decoder, on GPUs and for decoders of 8B parameters. Our port has forward-pass parity with it and serves its vectors through our own engine; what it cannot have on a CPU is the pre-training (3.3 million rows in two stages). The comparison of the two code bases explained our one failed module: a learning rate three times the reference's at a quarter of its batch, no dropout, and checkpoint selection by validation loss, which rewarded a module that had learnt a constant. The reference has no test for that failure; the collapse statistics and the activation guard of Section IX-D are ours. Memorilla has no run-time memory management (no write, forget or consolidation); that role is taken here by the note memory of Section IX-B.

DeepSeek Harness [57] is a ReAct loop over an event-sourced session log from which any prompt can be rebuilt

TABLE XVI
MEASURED AND NOT ADOPTED.

Idea	Measurement	Why
Speculative decoding with a 0.6B draft model [61]	6.5 against 7.3 tokens/s	the draft model competes for the same cores and bandwidth
EAGLE-3 speculative heads [64]	System 1 1.04× greedy, 0.95× at production temperature; System 2 0.99×; about half the drafted tokens accepted	verifying a draft costs real compute on a CPU
n-gram self-speculation	at most 1.12× (code repair), down to 0.84× (JSON)	as above
Flash attention, 8-bit cache, micro-batch, polling flags	no consistent gain; per-round ratios swing 0.07× to 3.8× with the load	inconclusive on a shared host
1-bit-class weights (LittleBit [70]), post-training conversion at 0.98 bits	0 of 12 exact-answer prompts against 9 for the dense model; decode $+5\,\%$	quality needs quantisation-aware training on a GPU
Concurrent hedging on System 2	4/8 and 4/7 verified against 4/8 and 3/8 sequential; $+44$ to $+87\,\%$ time on easy items	a batch at repair depth costs 2.2 to 3.4 single attempts (Proposition 16)
A larger System 1 (Qwen3.5-2B)	+17 points, 95% CI [+7, +30], $n = 30$; 3.7 times slower per task	latency rule (within 1.3 times) not met
A judge on the quick model	accepts a quarter of seeded wrong answers	not a soft verifier in the sense of Theorem 1
Lowest-window log-probability as the fixed rule's signal	AUROC 0.581 [0.488, 0.698] on GSM8K drafts	exact to act on (Proposition 11), too weak to use
Certified bins as the deciding rule at $n = 203$	keeps 54.5 % at 9.9 % wrong against 78.2 % at 10.1 % for the fixed rule	bounds too wide at this ledger size
A larger embedder (Qwen3-Embedding-4B)	alone, reworded R@1 0.25 to 0.45; in the hybrid, pooled MRR 0.787 against 0.804 for the 0.6B model	four times the embedding time, 2.5 times the storage
Convex score fusion	pooled MRR 0.387 against 0.388 for reciprocal rank	better on quoted, worse on reworded questions
BM25F field weights from 0 to 3	within one question of flat weights on every set	inert
Feedback terms from the first passages	cost eight quoted questions their first place	drift
PageRank arm in the default search	first eight passages unchanged on every set; reworded R@20 0.329 to at most 0.377	needs a reranker to use what it finds below rank eight
Model-written query expansion	8 s a question; -0.03 MRR on quoted questions	cost and mixed effect
Reranker, near-duplicate removal, AND-first search	answer already in the hybrid top four for 19 of 20 and 57 of 70 questions	nothing to gain on these sets
Memory tokens without pre-training	0/20 alone; 17/20 with passages against 18/20 for passages	Section X-F
A difficulty signal from the question alone (nearest labelled questions; prototype banks [49])	AUC for a wrong draft over 810 labelled drafts: domain 0.566, banks 0.573, nearest questions 0.615, the draft's own features 0.682, both together 0.690	the question adds 0.008 to what the draft already says; skipping the draft would touch 12 of 810 requests
Sentences of six passages packed into a block of 2,400 characters, headers counted, in place of four whole passages	prompts 25 % shorter and answers 20 to 26 % sooner (42 of 50 pairs); 39 of 50 right against 42 ($p = 0.38$)	the answers lost took a number from the wrong excerpt; kept as an option
Cache-blocking the prompt's weight products; a smaller prompt chunk	within $\pm 3\,\%$ once the benchmark was warmed up	reading a prompt is bound by the kernel's arithmetic, not by the caches
A meaning weight of 0.3 or 0.5 with sentence windows	reworded MRR 0.201 to 0.216 and 0.228; quoted 0.922 to 0.867 and 0.857	the quoted questions pay for it
Sentence windows counted in full ($m = 1$)	reworded MRR 0.182 against 0.201 at $m = 0.5$	a short window outranks the passage that answers
Cutting the passages of a prompt at the largest drop in similarity	3.90 to 3.77 passages a prompt (quoted), 3.33 to 2.99 (reworded); the answering passage left one prompt	a tenth of a passage saved for an answer lost

byte for byte. A run ends on the model's own claim; it has no verifier, no search over candidates, no rollback and no step limit, and its sandbox does not unshare the network. From it we took the anchored token meter, pruning before compaction, the refusal of an edit from a truncated reply and the repeat-attempt notice. It remains ahead on byte-exact prompt replay, subagents at scale and browser tools.

OntoSmith [76] builds a governed knowledge graph under a human-approved, pinned ontology with deterministic gates and a published precision record. Its served retrieval path has no BM25, fixed windows without overlap and no rank fusion, where ours had all three. What it does better, and what we took, is the role of the graph: the graph *supplies candidates* to the ranking instead of overriding it; an ontology stored as data; and a quote for every relation.

Measured side by side. The benchmarks of Semantic Router that compile offline were run interleaved with Rust counterparts, five rounds, median (Table XVII). Two results go against a simple reading. Their hand-written AVX2 assembly is 1.3 to 1.5 times faster than our intrinsics for long dot products; the difference is 50 to 60 ns on a vector scan that costs 18 to 22 ms a query. And the per-request cost of both signal layers is negligible beside one model call, so what separates the projects is how an answer comes to be accepted, not these numbers.

TABLE XVII
PER-REQUEST WORK, FIVE INTERLEAVED ROUNDS (LOAD 24 TO 29). THE ROUTING ROW IS NOT THE SAME COMPUTATION (THEIR 30 KEYWORD RULES AGAINST OUR DOMAIN CLASSIFIER); THE COMPRESSION ROWS ARE; THE LAST ROW COMPARES HOST OVERHEAD ONLY, AGAINST THE OTHER PROJECT'S OWN BUDGET.

Workload	Other	Nafs-Aql	Ratio
Routing signal, 2 KB prompt	197.7 µs	2.2 µs	90×
Extractive compression, 500 tok.	745 µs	310 µs	2.4×
Extractive compression, 2K tok.	5.07 ms	1.16 ms	4.4×
Extractive compression, 16K tok.	50.3 ms	8.96 ms	5.6×
Dot product, $n = 768$, AVX2	99 ns	145 ns	0.68×
Dot product, $n = 1024$, AVX2	135 ns	199 ns	0.68×
Dot product, $n = 1024$, portable	1.20 µs	1.21 µs	0.99×
Agent host overhead, 100 turns	budget 900 ms	1.75 ms	–

XII. LIMITATIONS

Indicative results. Every measurement comes from one shared four-core machine with tens to a few hundred items per comparison. Verified counts that differ by one or two tasks are within sampling noise (the same ten agent tasks verified four and five times in two rounds of the same arm). A release-scale evaluation needs thousands of paired items and a

quiet or faster host. The formal results hold under their stated assumptions, several of which (conditional independence of verifiers, a Beta prior on acceptance, homogeneous refinement dynamics, concave budget returns) are modelling choices.

The calibrated gate is not yet in force. The ledger holds 1,294 labelled drafts of the 2,900 its promotion rule asks for, and only one of five bins is certified at the default bound, so the fixed rule decides. Token confidence is a weak signal for these models on mathematics; a draft whose samples all repeat the same wrong answer is kept.

Verifiers are partial. Only code, mathematics, structured output and Lean statements have exact verifiers; everything else is at best a consensus or judge assurance, which Theorem 6 shows has a floor. Test-based verification can be gamed by overfitting to visible tests, which is why tests are hidden and the code verifier's false-accept rate is measured by mutation. The 4B model does not write valid Lean 4 proofs, so Lean tasks abstain; Mathlib is not installed.

Shared weights. The fast and slow paths come from one model family and make correlated errors; the collision term of Proposition 4 is not small a priori, and the judge shares the bias, which is why exact verifiers carry the safety case.

The CPU. A deliberate answer takes minutes. The default engine's server lacks grammars, n-choice requests and raw continuation, so the rescue, the cascade and the adaptive exit run at full strength only on the llama.cpp backend. Engine-flag studies were inconclusive under shared load. Distillation of the quick model and pre-training of the memory module need a GPU and were not run.

Knowledge. Questions worded differently from the document are found in the first eight passages only a quarter of the time. The remedies (a reranker, a stronger embedder, questions written for each passage at indexing time) need larger models or hours of the quick model. No public retrieval benchmark was run, so there is no comparison with published systems. The model-built graph costs a model call per passage, and community summaries are not built. The LaTeX reader leaks macro names inside mathematics. At 50,000 passages search by words through the store is slow under load. The support check compares words and numbers: it flags a sentence that is not in the passages, and it passes a wrong sentence assembled from the passages' own words. The rule for follow-up questions reads one previous turn and is written for English.

Memory. Memory tokens do not work without pre-training, and no module trained with the corrected recipe exists yet. The memory of earlier conversations is one person's: outside chat rooms the system has no accounts, so everyone who can reach an installation shares it. Its gate is a word list, so a question that needs an earlier conversation without saying so is answered without it.

The agent. Tasks are single-file functions; there are no subagents, no plan mode and no byte-exact replay of a prompt from the trace. Grammar constraints fix the syntax of an action, not its sense.

Security and privacy. The system executes model-written code in a sandbox and consumes untrusted text; prompt injection through retrieved passages or tool output is a first-class threat [78], [79]. Retrieved content is delimited as data, relations enter a prompt only with a verbatim quote, and observed content never changes controller parameters; classifiers for jailbreaks and personal data are pattern-based, not trained. Conversation traces feed the ledger; using them for training requires consent, retention limits and redaction.

Faithfulness. Separating the reasoning from the answer does not make the reasoning faithful [87], [88]; the trace is evidence for debugging, not an explanation to the user. The system makes no claim of consciousness or moral standing; its name is an analogy (Section I).

XIII. Conclusion

Nafs-Aql shows that a useful dual-process assistant fits on four CPU cores: a small Qwen model answers at once, a gate derived from expected loss decides when deliberation pays, a reasoning model deliberates under a budget, exact verifiers decide what counts as known, and an audited ledger keeps the system honest about itself. The mathematics fixed the design before the measurements did: precision on checkable tasks is the verifier's, not the model's; agreement is a flag and never proof; every loop is bounded; and a loop that accepts only on a verifier's pass is stable whatever the model does inside it.

Three lessons recur across the measurements. First, on a CPU the gains are in not repeating work. Decode runs at the memory roofline, so more threads, speculation and hedging did not pay, while prefix reuse, shared prefill across candidates, one pass over the weights for several users and an in-process index did, by factors from 1.2 to three orders of magnitude. Second, what makes small models usable as agents is discipline imposed by the harness: an action language the engine enforces, a history that is never rewritten, an exact verifier after every change and a hard budget. Third, cheap candidates from the quick model, judged by an exact verifier before the expensive model is consulted, are the best use of idle cores, in the chat path and in the agent alike.

The open work is stated in Section XII: a ledger large enough to promote the calibrated gate, retrieval for reworded questions, evaluation at release scale and on public benchmarks, and the training steps that need a GPU. The system, its tests and every stored run behind the figures in this paper are available with the source.

References

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[2] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2203.11171

[3] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv preprint (Aug. 2024) titled "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters". [Online]. Available: https://arxiv.org/abs/2408.03314

[4] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[5] X. Chen, J. Xu, T. Liang, Z. He, J. Pang, D. Yu, L. Song, Q. Liu, M. Zhou, Z. Zhang, R. Wang, Z. Tu, H. Mi, and D. Yu, "Do NOT think that much for 2+3=? On the overthinking of o1-like LLMs," *arXiv preprint arXiv:2412.21187*, 2024. [Online]. Available: https://arxiv.org/abs/2412.21187

[6] J. Zhang, N. Lin, L. Hou, L. Feng, and J. Li, "AdaptThink: Reasoning models can learn when to think," *arXiv preprint arXiv:2505.13417*, 2025. [Online]. Available: https://arxiv.org/abs/2505.13417

[7] D. Kahneman, *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux, 2011.

[8] J. S. B. T. Evans and K. E. Stanovich, "Dual-process theories of higher cognition: Advancing the debate," *Perspectives on Psychological Science*, vol. 8, no. 3, pp. 223–241, 2013.

[9] S. A. Sloman, "The empirical case for two systems of reasoning," *Psychological Bulletin*, vol. 119, no. 1, pp. 3–22, 1996.

[10] K. E. Stanovich and R. F. West, "Individual differences in reasoning: Implications for the rationality debate?" *Behavioral and Brain Sciences*, vol. 23, no. 5, pp. 645–665, 2000.

[11] A. Shenhav, M. M. Botvinick, and J. D. Cohen, "The expected value of control: An integrative theory of anterior cingulate cortex function," *Neuron*, vol. 79, no. 2, pp. 217–240, 2013.

[12] F. Lieder and T. L. Griffiths, "Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources," *Behavioral and Brain Sciences*, vol. 43, p. e1, 2020.

[13] S. Russell and E. Wefald, "Principles of metareasoning," *Artificial Intelligence*, vol. 49, no. 1-3, pp. 361–395, 1991.

[14] J. I. Gold and M. N. Shadlen, "The neural basis of decision making," *Annual Review of Neuroscience*, vol. 30, pp. 535–574, 2007.

[15] A. Wald, "Sequential tests of statistical hypotheses," *The Annals of Mathematical Statistics*, vol. 16, no. 2, pp. 117–186, 1945.

[16] A. Wald and J. Wolfowitz, "Optimum character of the sequential probability ratio test," *The Annals of Mathematical Statistics*, vol. 19, no. 3, pp. 326–339, 1948. [Online]. Available: https://www.projecteuclid.org/euclid.aoms/1177730197

[17] P. I. Frazier and A. J. Yu, "Sequential hypothesis testing under stochastic deadlines," in *Advances in Neural Information Processing Systems 20 (NIPS 2007)*, 2007, pp. 465–472. [Online]. Available: https://proceedings.neurips.cc/paper/2007/hash/9c82c7143c102b71c593d98d96093fde-Abstract.html

[18] J. Drugowitsch, R. Moreno-Bote, A. K. Churchland, M. N. Shadlen, and A. Pouget, "The cost of accumulating evidence in perceptual decision making," *Journal of Neuroscience*, vol. 32, no. 11, pp. 3612–3628, 2012.

[19] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, "Large language monkeys: Scaling inference compute with repeated sampling," *arXiv preprint arXiv:2407.21787*, 2024. [Online]. Available: https://arxiv.org/abs/2407.21787

[20] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021. [Online]. Available: https://arxiv.org/abs/2110.14168

[21] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2305.20050

[22] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025. [Online]. Available: https://doi.org/10.1038/s41586-025-09422-z

[23] OpenAI, A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky *et al.*, "OpenAI o1 system card," *arXiv preprint arXiv:2412.16720*, 2024. [Online]. Available: https://arxiv.org/abs/2412.16720

[24] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto, "s1: Simple test-time scaling," *arXiv preprint arXiv:2501.19393*, 2025. [Online]. Available: https://arxiv.org/abs/2501.19393

[25] A. Yang, A. Li, B. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[26] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[27] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, arXiv version (2303.11366) additionally lists Edward Berman as an author. [Online]. Available: https://arxiv.org/abs/2303.11366

[28] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.01798

[29] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang, "When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1417–1440, 2024. [Online]. Available: https://arxiv.org/abs/2406.01297

[30] G. Fang, X. Ma, and X. Wang, "Thinkless: LLM learns when to think," *arXiv preprint arXiv:2505.13379*, 2025. [Online]. Available: https://arxiv.org/abs/2505.13379

[31] Y. He, X. Ding, B. Cai, Y. Zhang, K. Xiong, Z. Sun, B. Qin, and T. Liu, "Self-route: Automatic mode switching via capability estimation for efficient reasoning," *arXiv preprint arXiv:2505.20664*, 2025. [Online]. Available: https://arxiv.org/abs/2505.20664

[32] B. Hou, Y. Zhang, J. Ji, Y. Liu, K. Qian, J. Andreas, and S. Chang, "ThinkPrune: Pruning long chain-of-thought of LLMs via reinforcement learning," *arXiv preprint arXiv:2504.01296*, 2025. [Online]. Available: https://arxiv.org/abs/2504.01296

[33] C. Yang, Q. Si, Y. Duan, Z. Zhu, C. Zhu, Q. Li, M. Chen, Z. Lin, and W. Wang, "Dynamic early exit in reasoning models," *arXiv preprint arXiv:2504.15895*, 2025. [Online]. Available: https://arxiv.org/abs/2504.15895

[34] D. Arora and A. Zanette, "Training language models to reason efficiently," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. [Online]. Available: https://arxiv.org/abs/2502.04463

[35] P. Aggarwal, A. Madaan, Y. Yang, and Mausam, "Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs," in *Proceedings of EMNLP 2023*, 2023, pp. 12 375–12 396. [Online]. Available: https://aclanthology.org/2023.emnlp-main.761

[36] Y. Li, P. Yuan, S. Feng, B. Pan, X. Wang, B. Sun, H. Wang, and K. Li, "Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning," in *International Conference on Learning Representations (ICLR)*, 2024.

[37] Y. Fu, X. Wang, Y. Tian, and J. Zhao, "Deep think with confidence," *arXiv preprint arXiv:2508.15260*, 2025. [Online]. Available: https://arxiv.org/abs/2508.15260

[38] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, "Hybrid LLM: Cost-efficient and quality-aware query routing," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2404.14618

[39] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs from preference data," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv version titled "RouteLLM: Learning to Route LLMs with Preference Data". [Online]. Available: https://arxiv.org/abs/2406.18665

[40] M. Yue, J. Zhao, M. Zhang, L. Du, and Z. Yao, "Large language model cascades with mixture of thoughts representations for cost-efficient reasoning," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.03094

[41] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023. [Online]. Available: https://arxiv.org/abs/2305.05176

[42] C. K. Chow, "On optimum recognition error and reject tradeoff," *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970.

[43] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

[44] D. Madras, T. Pitassi, and R. Zemel, "Predict responsibly: Improving fairness and accuracy by learning to defer," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018. [Online]. Available: https://arxiv.org/abs/1711.06664

[45] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," in *Proceedings of the 37th International*

Conference on Machine Learning (ICML), 2020. [Online]. Available: https://arxiv.org/abs/2006.01862

[46] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.04599

[47] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez *et al.*, "Language models (mostly) know what they know," *arXiv preprint arXiv:2207.05221*, 2022. [Online]. Available: https://arxiv.org/abs/2207.05221

[48] K. Tian, E. Mitchell, A. Zhou, A. Sharma, R. Rafailov, H. Yao, C. Finn, and C. D. Manning, "Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14975

[49] vLLM Project, "vLLM Semantic Router: an intelligent routing layer for mixture-of-models," https://github.com/vllm-project/semantic-router, 2026, commit 029bf54ed.

[50] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[51] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.

[52] T. Lindenbauer, I. Slinko, L. Felder, E. Bogomolov, and Y. Zharov, "The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management," *arXiv preprint arXiv:2508.21433*, 2025.

[53] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.

[54] R. Xin *et al.*, "BFS-Prover: Scalable best-first tree search for LLM-based automatic theorem proving," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, arXiv:2502.03438.

[55] Open Math OS, "Open math harness (nucleus), release 0.41.0," https://github.com/openmathmodel/open-math-harness, revision 687f2a2, 2026, apache-2.0.

[56] OpenAI, "Codex CLI (codex-rs)," https://github.com/openai/codex, revision 970111f6, 2026, apache-2.0.

[57] DeepSeek, "DeepSeek Harness (dsh): an event-sourced agent harness," https://github.com/deepseek-ai/deepseek-harness, 2026, commit fcc0ad4621.

[58] E. Glukhov, M. Conti, E. Bogomolov, Y. Golubev, and A. Bezzubov, "Diff-XYZ: A benchmark for evaluating diff understanding," *arXiv preprint arXiv:2510.12487*, 2025.

[59] Aider, "Edit formats," https://aider.chat/docs/more/edit-formats.html, 2024, whole-file, diff and udiff formats and their per-model reliability.

[60] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen, "Let me speak freely? A study on the impact of format restrictions on performance of large language models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2024, arXiv:2408.02442.

[61] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. [Online]. Available: https://arxiv.org/abs/2211.17192

[62] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023. [Online]. Available: https://arxiv.org/abs/2302.01318

[63] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 5209–5235. [Online]. Available: https://arxiv.org/abs/2401.10774

[64] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 28 935–28 948. [Online]. Available: https://arxiv.org/abs/2401.15077

[65] G. Oliaro, Z. Jia, D. Campos, and A. Qiao, "SuffixDecoding: Extreme speculative decoding for emerging AI applications," *arXiv preprint arXiv:2411.04975*, 2024.

[66] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[67] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[68] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[69] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[70] B. Lee, D. Kim, Y. You, and Y. Kim, "Littlebit: Ultra low-bit quantization via latent factorization," *arXiv preprint arXiv:2506.13771*, 2025.

[71] J. Wei, S. Cao *et al.*, "T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge," *arXiv preprint arXiv:2407.00088*, 2024.

[72] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," *Proceedings of SIGIR*, 2009.

[73] Anthropic, "Introducing contextual retrieval," https://www.anthropic.com/news/contextual-retrieval, 2024.

[74] LlamaIndex, "LiteParse: fast, local document parsing with layout analysis," https://github.com/run-llama/liteparse, 2026, commit 0548949.

[75] ——, "semtools: semantic search and document parsing tools for the command line," https://github.com/run-llama/semtools, 2026, version 3.0.0, commit fe7f6d2.

[76] D. Lee, "Ontosmith: an ontology-driven knowledge-graph builder," https://github.com/Dongbumlee/ontosmith, revision b1007d6, 2026.

[77] Snap Research and Stanford SNAP Group, "Memorilla: Latent semantic memory for a frozen language model," arXiv preprint arXiv:2610.21375, 2026.

[78] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90. [Online]. Available: https://arxiv.org/abs/2302.12173

[79] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS ML Safety Workshop*, 2022. [Online]. Available: https://arxiv.org/abs/2211.09527

[80] Y. S. Chow, H. Robbins, and D. Siegmund, *Great Expectations: The Theory of Optimal Stopping*. Boston, MA, USA: Houghton Mifflin, 1971, reissued by Dover, 1991, as *The Theory of Optimal Stopping*. Verified via HathiTrust record 000469250, accessed 2026-10-18.

[81] W. Hoeffding, "Probability inequalities for sums of bounded random variables," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 13–30, 1963.

[82] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[83] P. Massart, "The tight constant in the Dvoretzky-Kiefer-Wolfowitz inequality," *The Annals of Probability*, vol. 18, no. 3, pp. 1269–1283, 1990.

[84] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1-2, pp. 100–115, 1954.

[85] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," Automated Deduction – CADE 28, LNCS 12699, pp. 625–635, 2021.

[86] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021. [Online]. Available: https://arxiv.org/abs/2108.07732

[87] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.04388

[88] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. [Online]. Available: https://arxiv.org/abs/2202.03629